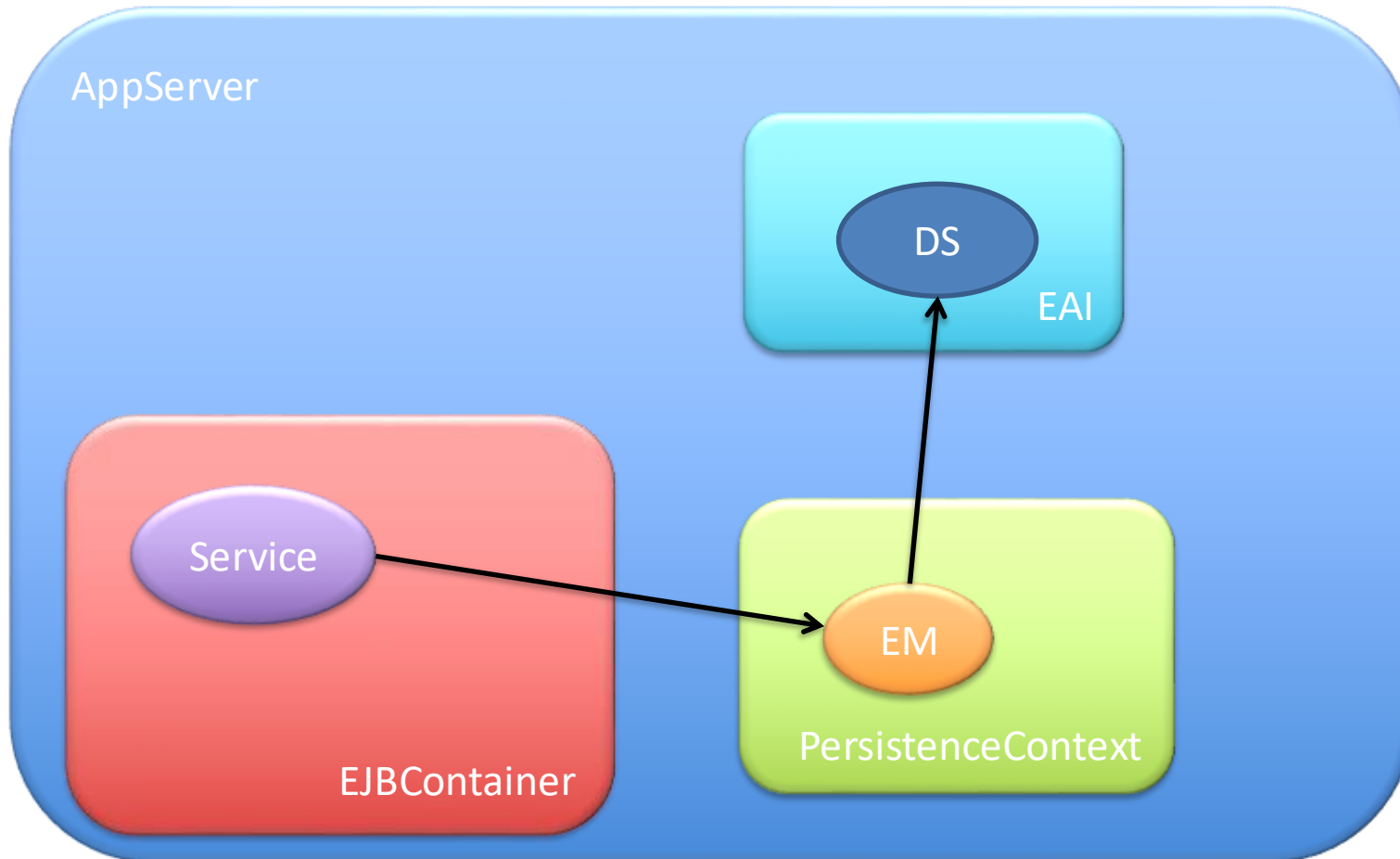
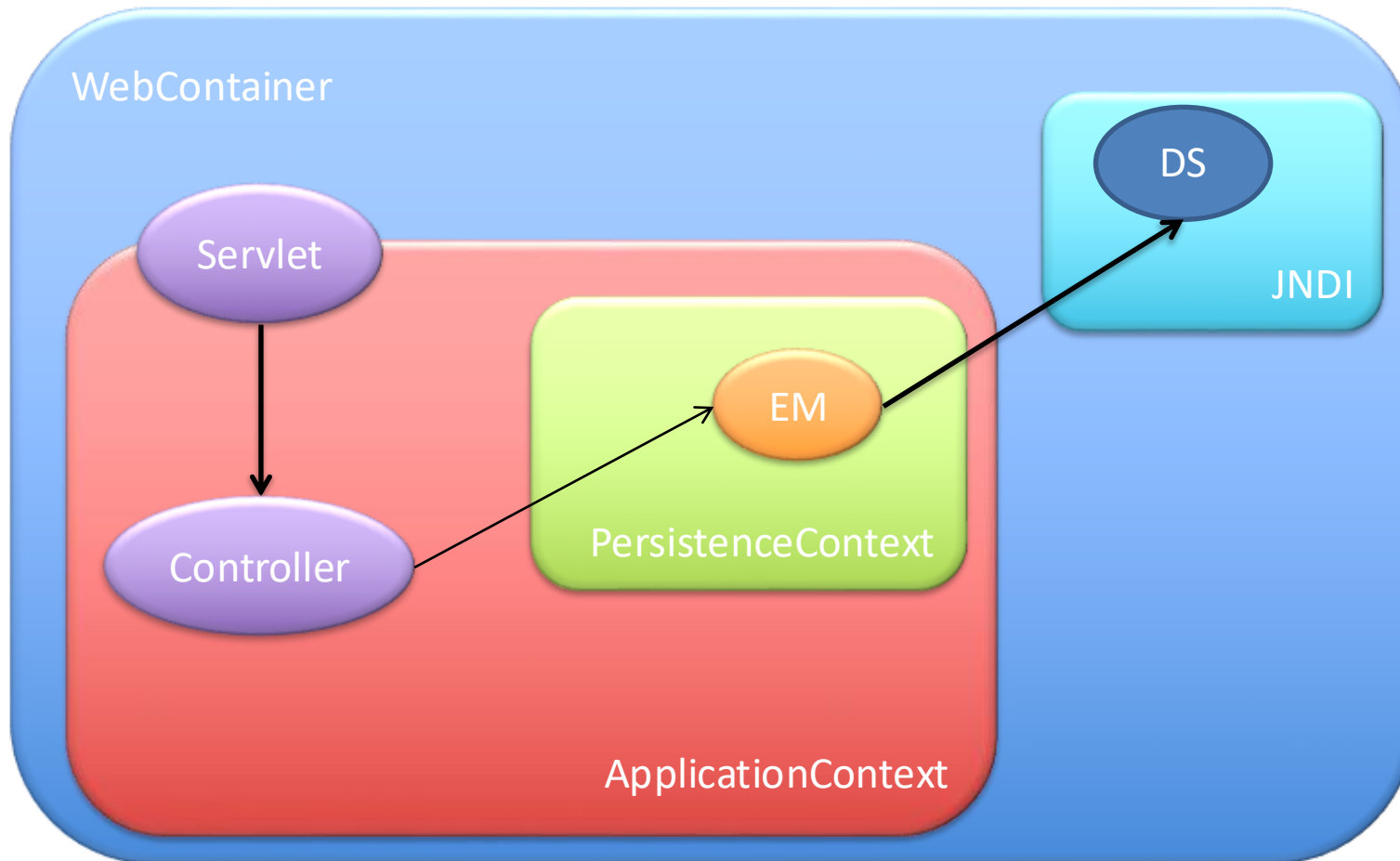


Entitäten mit der Java Persistence API

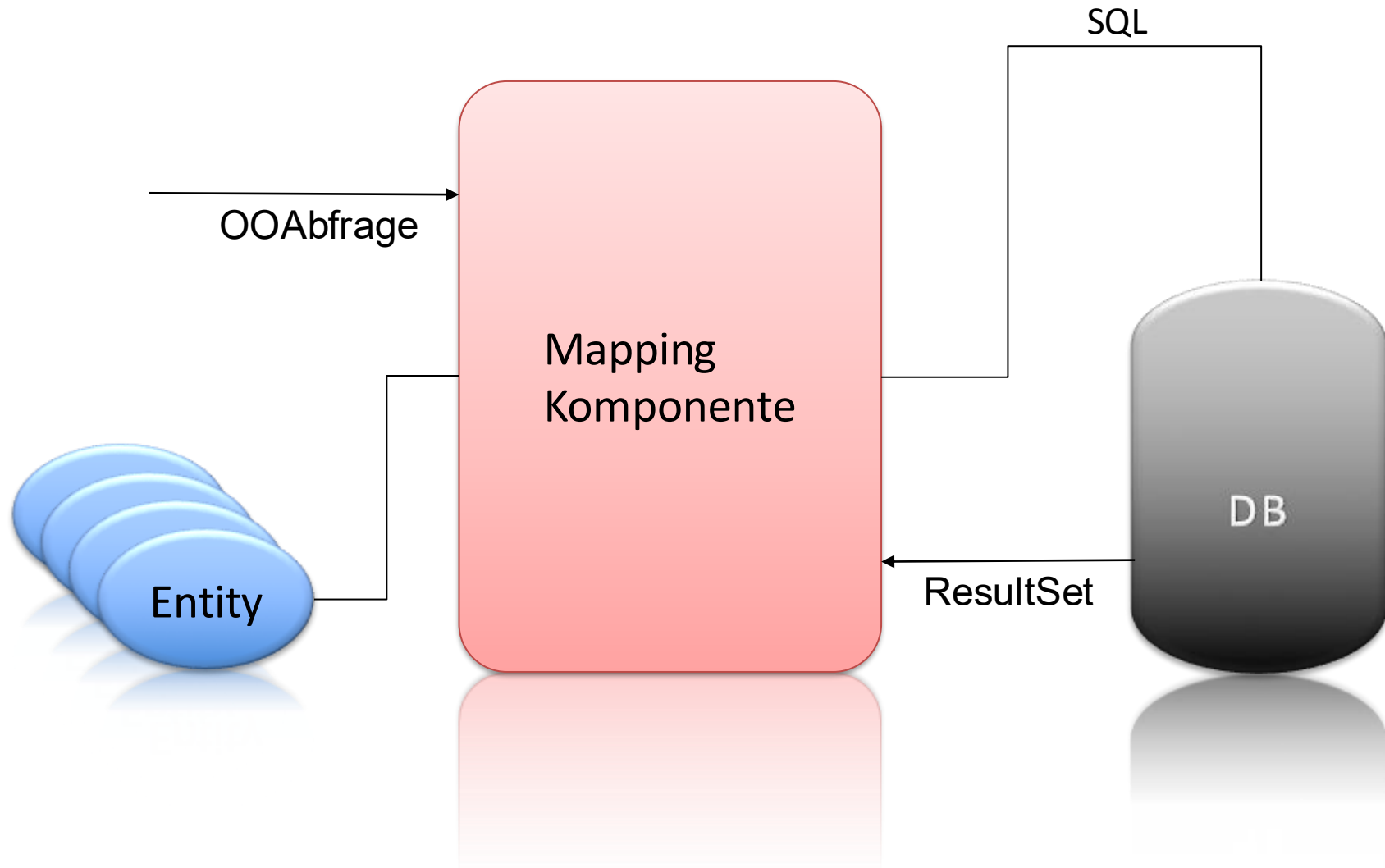
JEE World



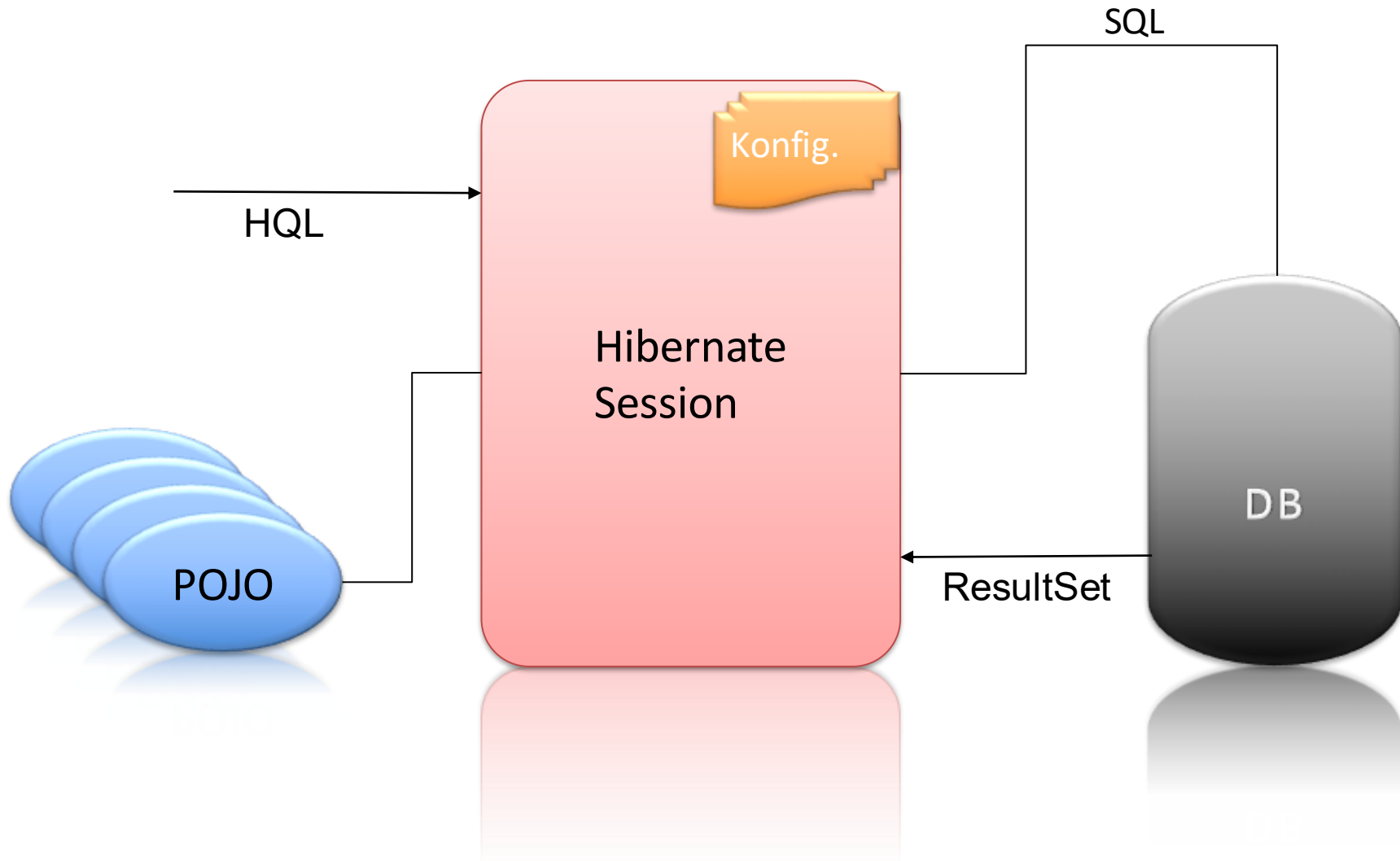


Spring on WEB

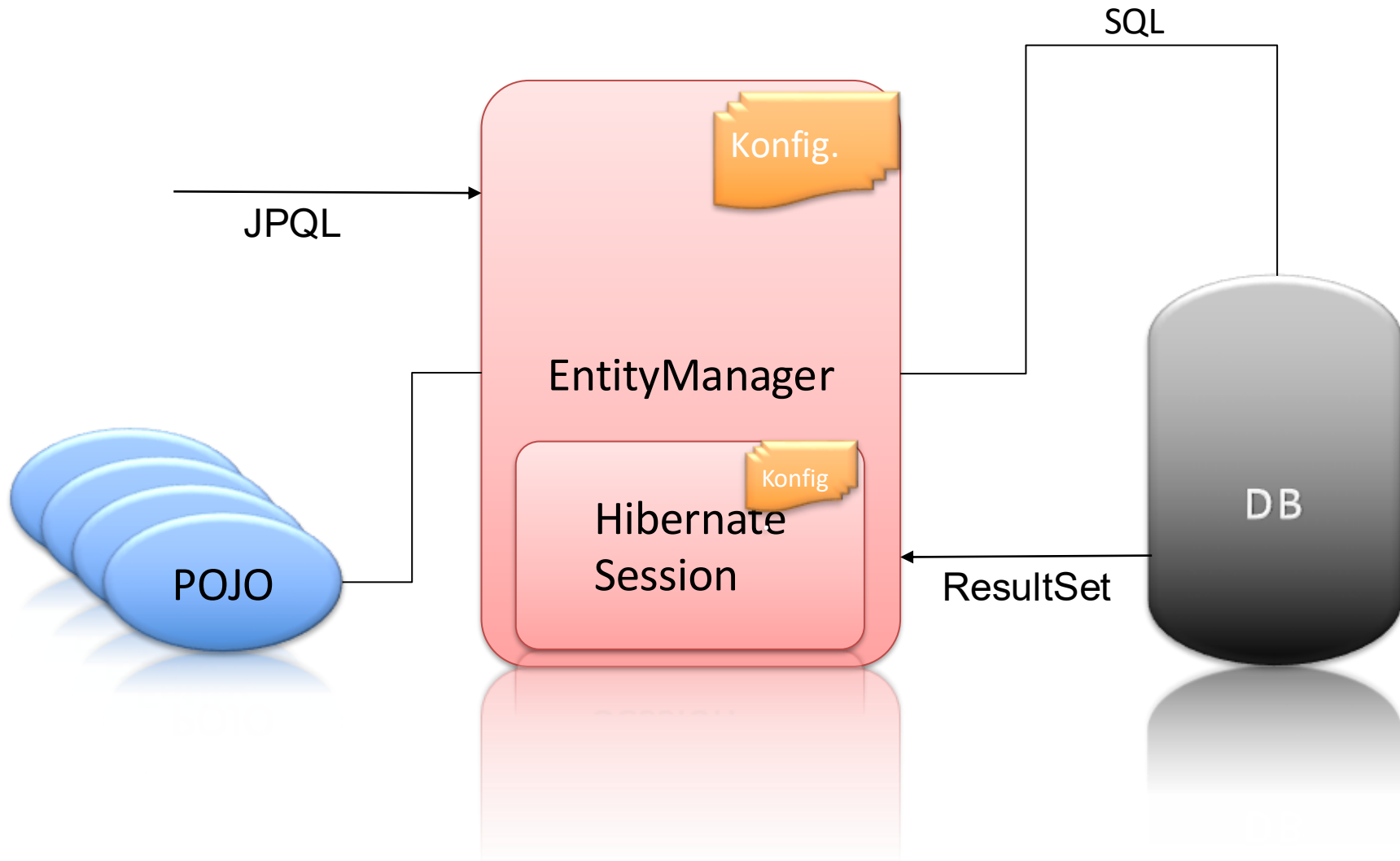
Zentrale Mapping Komponente

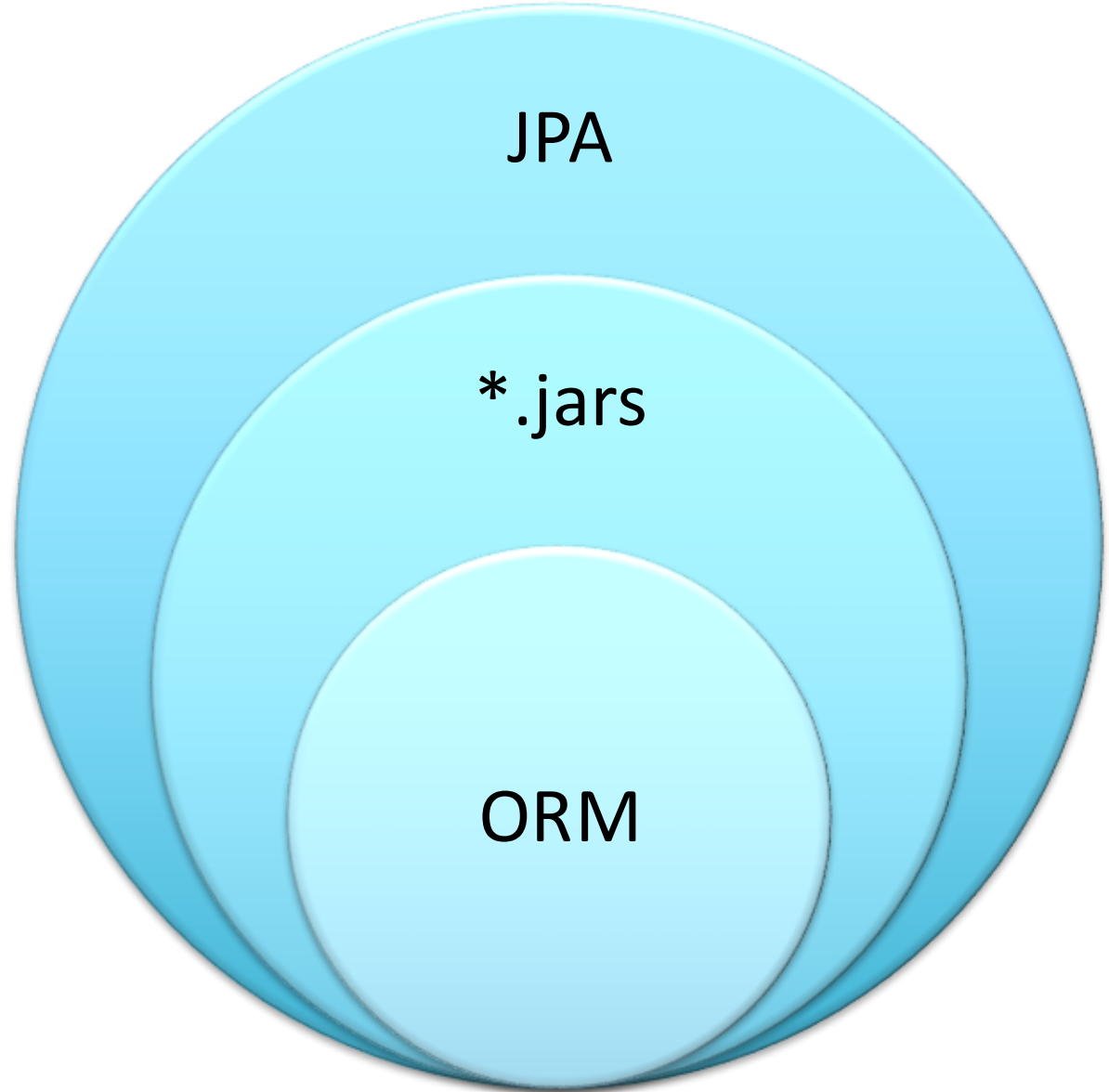


Hibernate Session



JPA





What developers **Need**

JBossTools



DALI

Hibernate
Tools

Standard RevEng

Entity Mapping

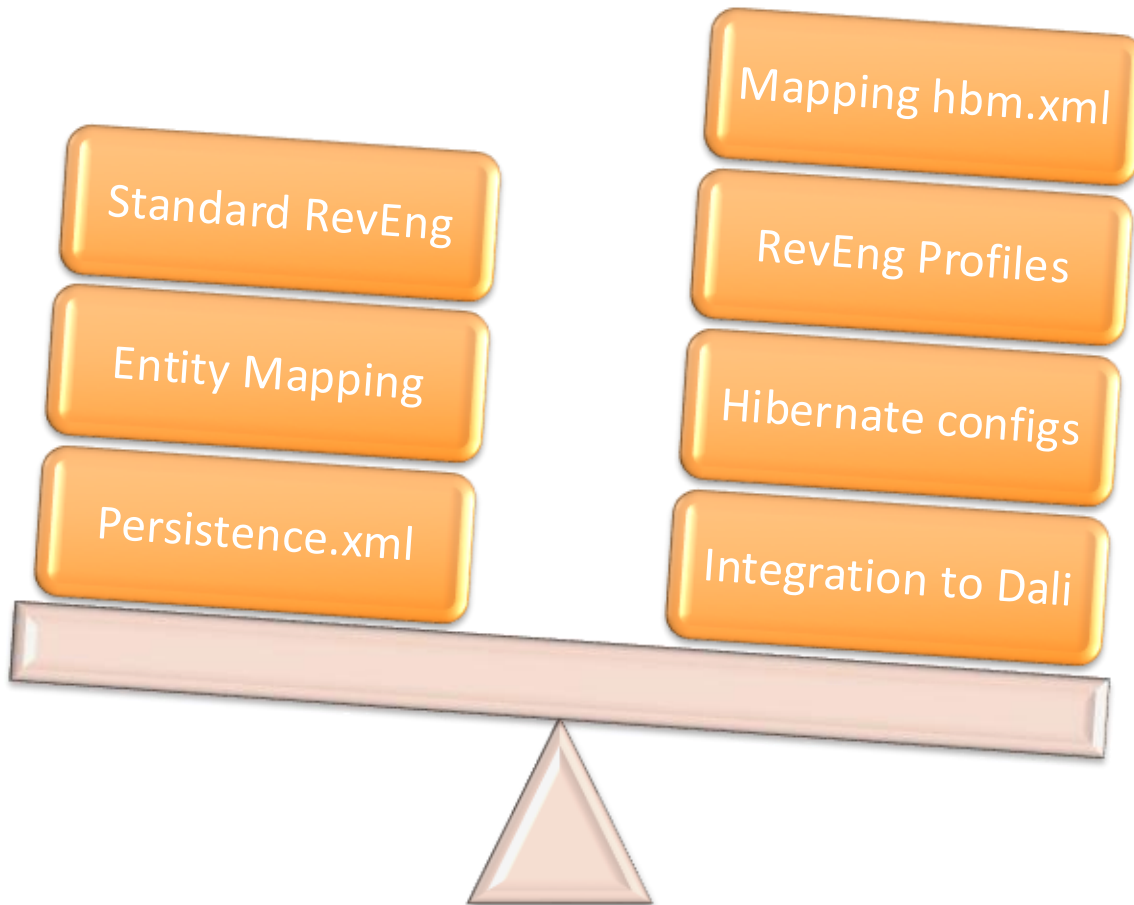
Persistence.xml

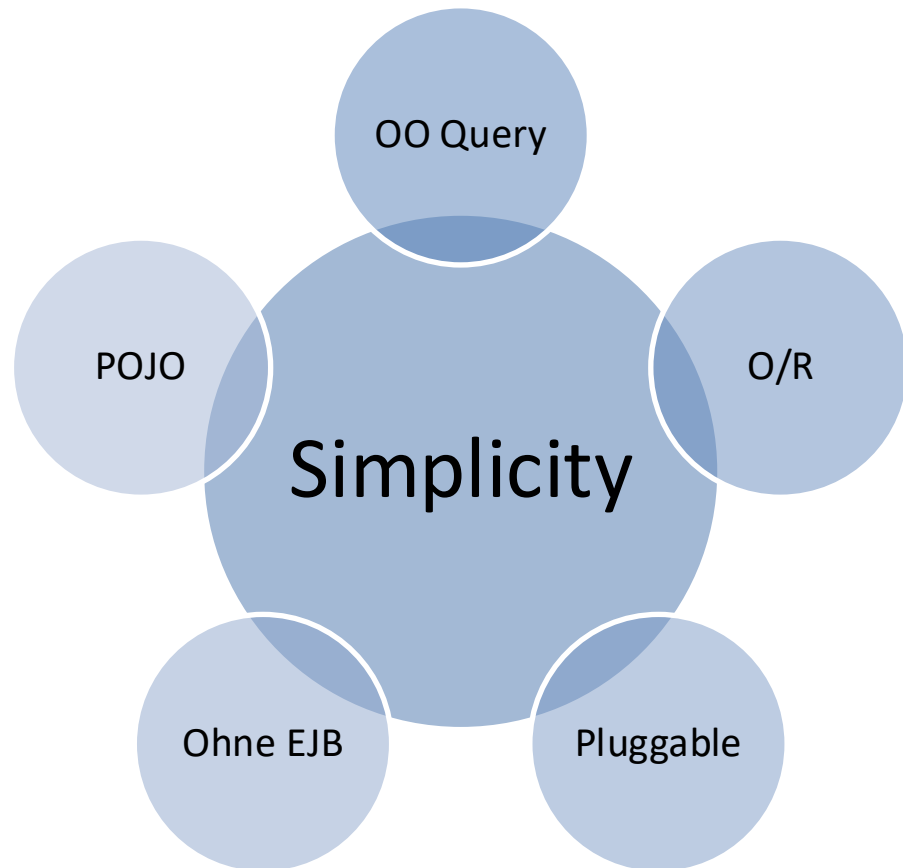
Mapping hbm.xml

RevEng Profiles

Hibernate configs

Integration to Dali





Ideen



HIBERNATE



eclipse

Ergebnis

Weniger Klassen und Interfaces

Keine längliche Deployment
Descriptors nötig

Configuration by Default

Kein Lookup Code

Unterstützt

- Vererbung
- Polymorphie und polymorphe Abfragen

Named Queries

Nutzbar außerhalb des EJB
Containers

- Testbarkeit

Pluggable Persistence Provider

JPA 1.0

```
@Entity
```

```
public class Adress {  
  
}
```

Grundlagen

Grundlagen Entity Manager

Zentrale Komponente für
Persistenz Funktionalität

Ziel:

- Entities sind POJOs

Aufgabe O/R Mapping

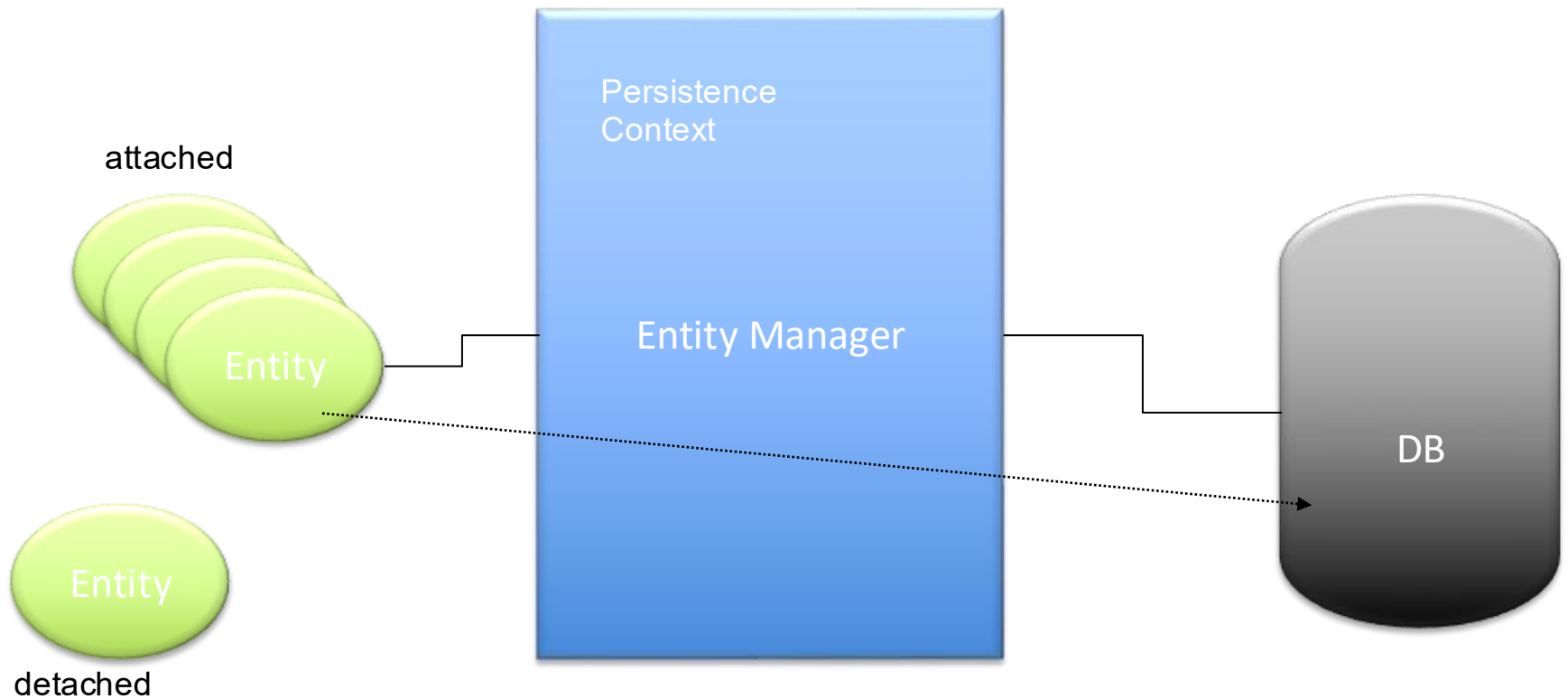
- Einfügen, Löschen, Suchen

Weitere Aufgaben

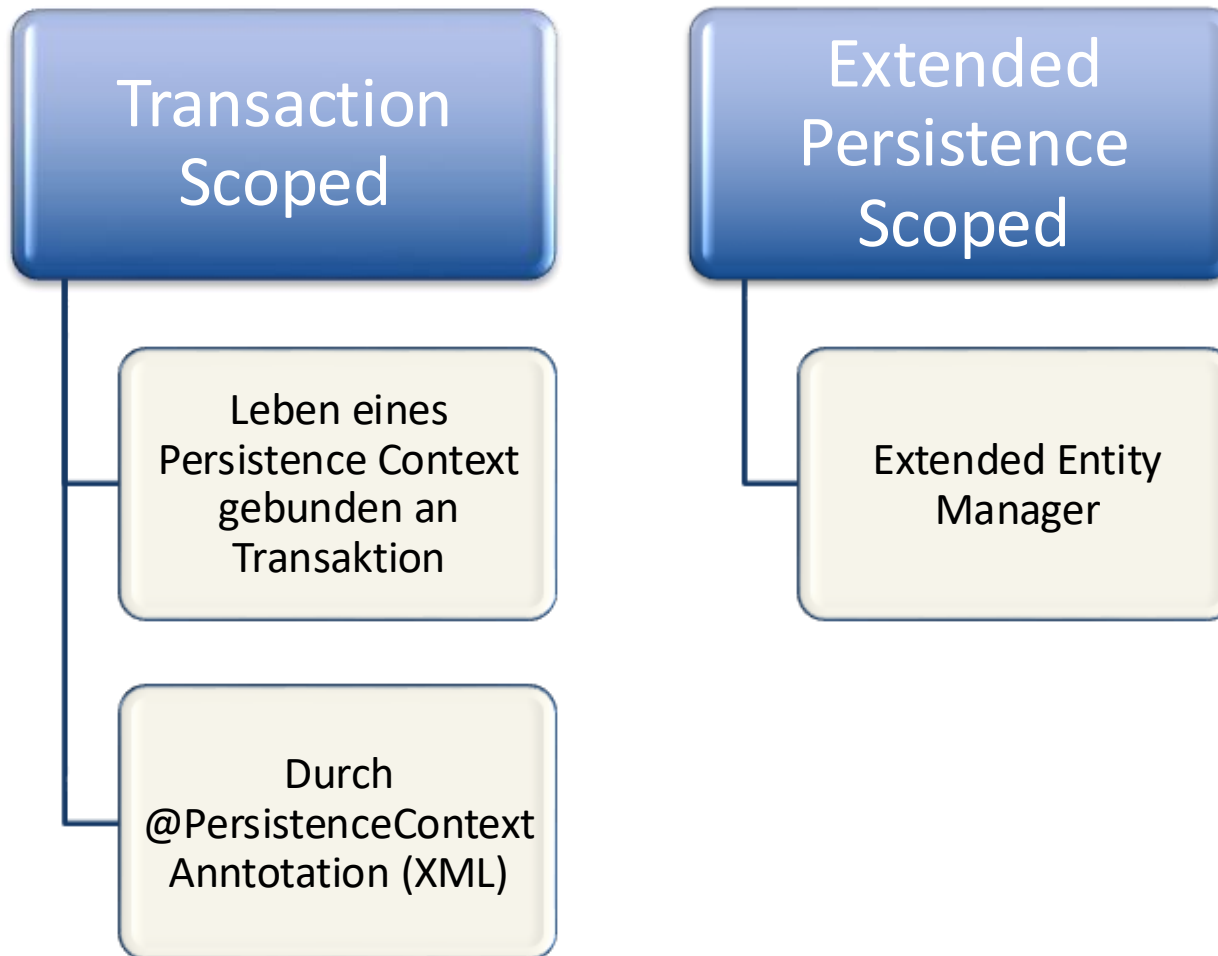
- z.B. Caching

Nicht an EJB Umgebung
gebunden

Entity Lebenszyklus

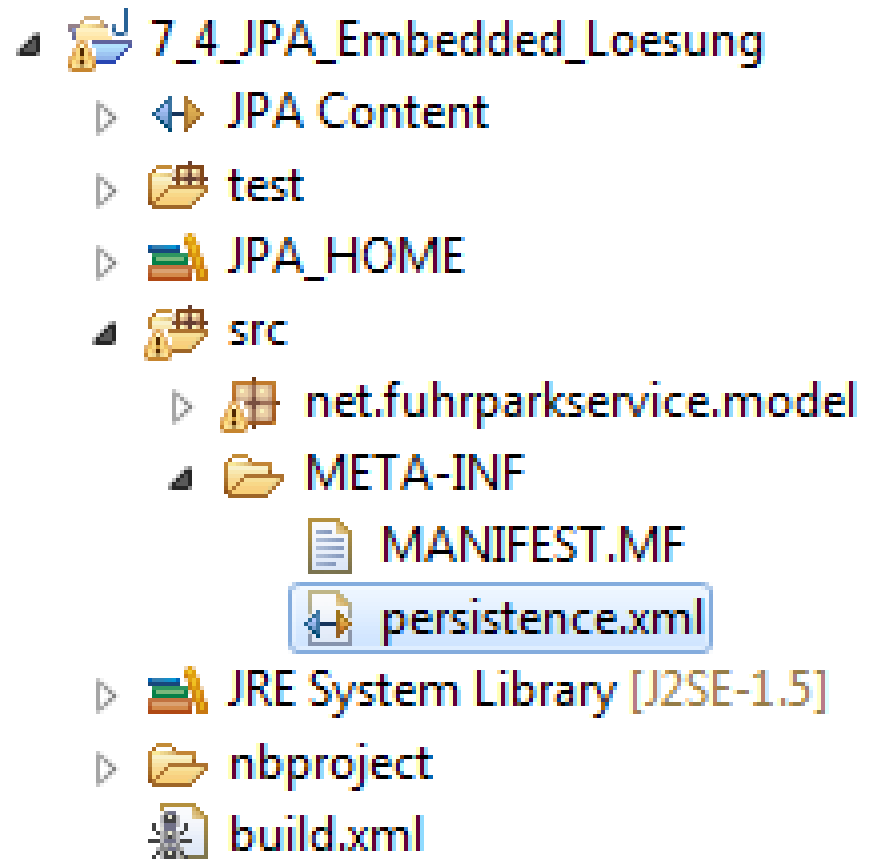


Persistence Context



Persistence.xml

- META-INF Verzeichnis
 - JAR
 - EJB JAR
 - WAR (WEB-INF/lib)
 - EAR/lib
- Identifiziert und Beschreibt n Persistence Units
- Auflistung / Suchen(Annotation) der Entities



Persistence Unit

```
<persistence version="1.0"
  xmlns="http://java.sun.com/xml/ns/persistence" ..
  xsi:schemaLocation="...">
  <persistence-unit name="projectUnit" transaction-
    type="RESOURCE_LOCAL">
    <provider>org.eclipse.persistence.
      jpa.PersistenceProvider</provider>
  </properties>
</persistence-unit>
</persistence>
```

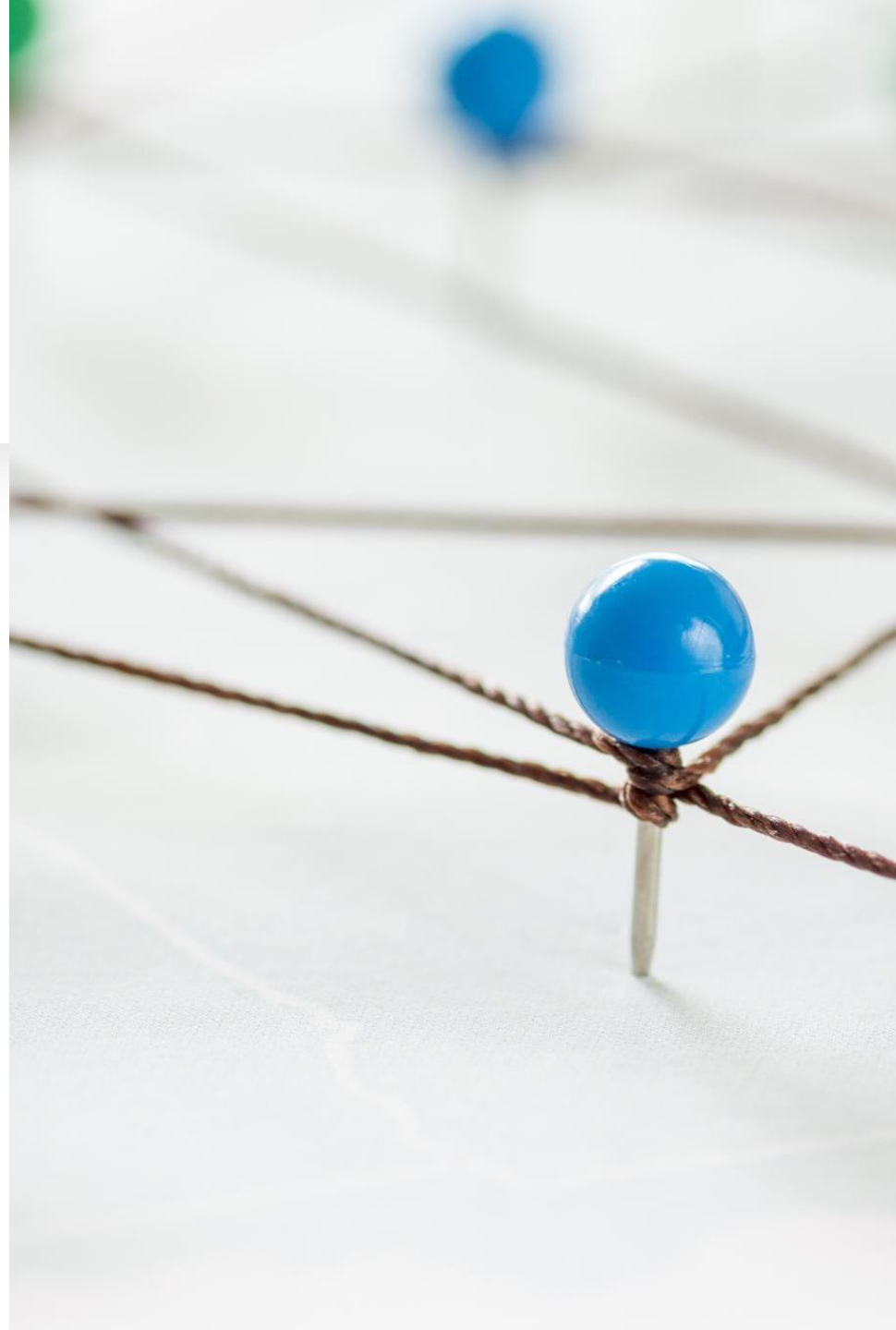
Entity Manager

- `persist(Object entity)`
- `find(Class <T> entityClass, Object key)`
- `getReference(Class <T> entityClass, Object key)`
 - throws `EntityNotFoundException`
- `merge(T entity)`
- `remove(Object entity)`
- `lock(Object entity, LockModeType lockMode)`
- `refresh(Object entity)`
- `contains(Object entity)`
- `clear()`

Entity Manager - 2

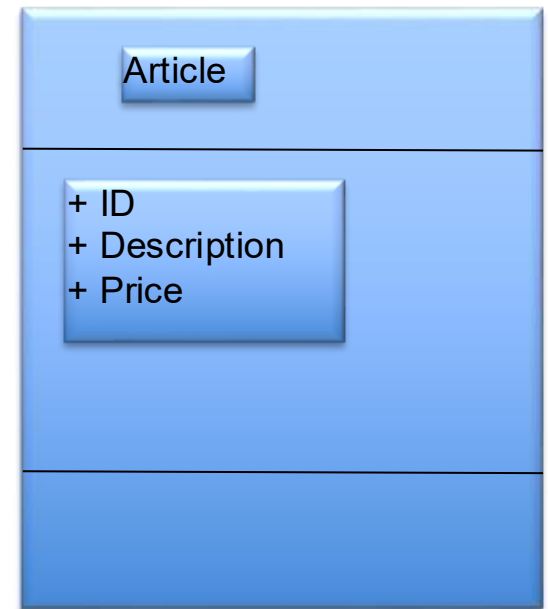
- `createQuery(String queryString)`
- `createNamedQuery(String name)`
- `createNativeQuery(String sqlString)`
- `createNativeQuery(String sqlString, String resultSetMapping)`
- `createNativeQuery(String sqlString, Class resultClass)`
- `getDelegate()`

- **Objekt Relationales Mapping**



OR Mapping

ID	Description	Price
1	Java ist auch eine Insel	49,99 €
2	Hibernate	39,99 €



Application



Database

<properties>

```
<property name="javax.persistence.jdbc.driver" value="org.apache.derby.jdbc.ClientDriver" />
<property name="eclipselink.target-database" value="Derby" />
<property name="javax.persistence.jdbc.url"
           value="jdbc:derby://localhost:1527/sun-appserv-samples;create=true" />
<property name="javax.persistence.jdbc.user" value="APP" />
<property name="javax.persistence.jdbc.password" value="a" />
```

<properties>

```
<property name="eclipselink.ddl-generation" value="drop-and-create-tables" />  
<property name="eclipselink.ddl-generation.output-mode"  
          value="database" />  
<property name="eclipselink.logging.level" value="FINEST" />  
<property name="eclipselink.logging.parameters" value="true"/>
```

<properties>

```
<property name="hibernate.hbm2ddl.auto" value="create" />  
<property name="hibernate.dialect"  
    value="org.hibernate.dialect.DerbyDialect" />
```



```
EntityManagerFactory managerFactory =  
    Persistence.createEntityManagerFactory("projectUnit");
```

```
EntityManager manager = managerFactory.createEntityManager();  
manager.isOpen();
```



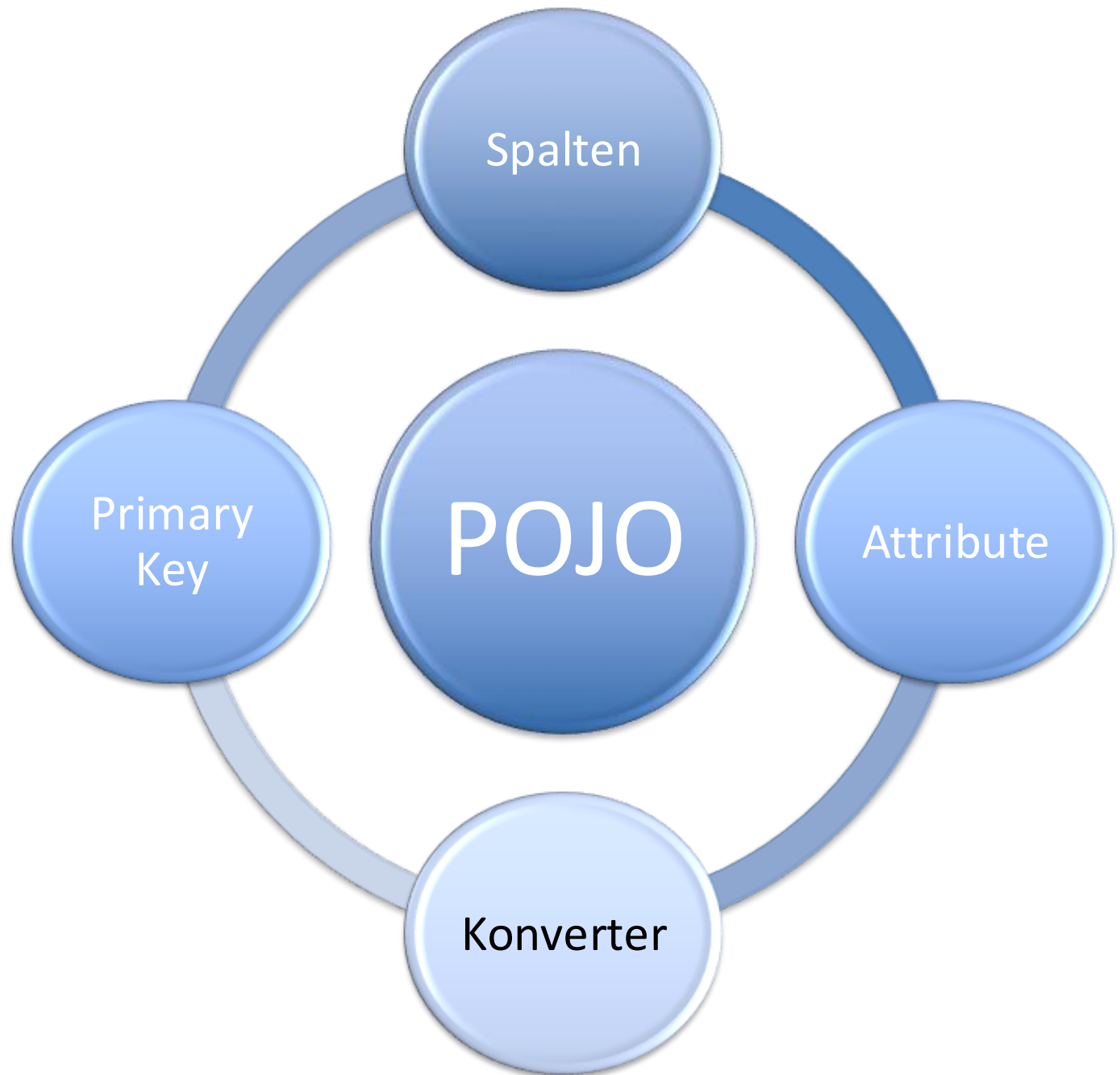
persistence.xml

```
<persistence version="1.0"  
xmlns="http://java.sun.com/xml/ns/persistence" ..>  
    <persistence-unit name="projectUnit"
```

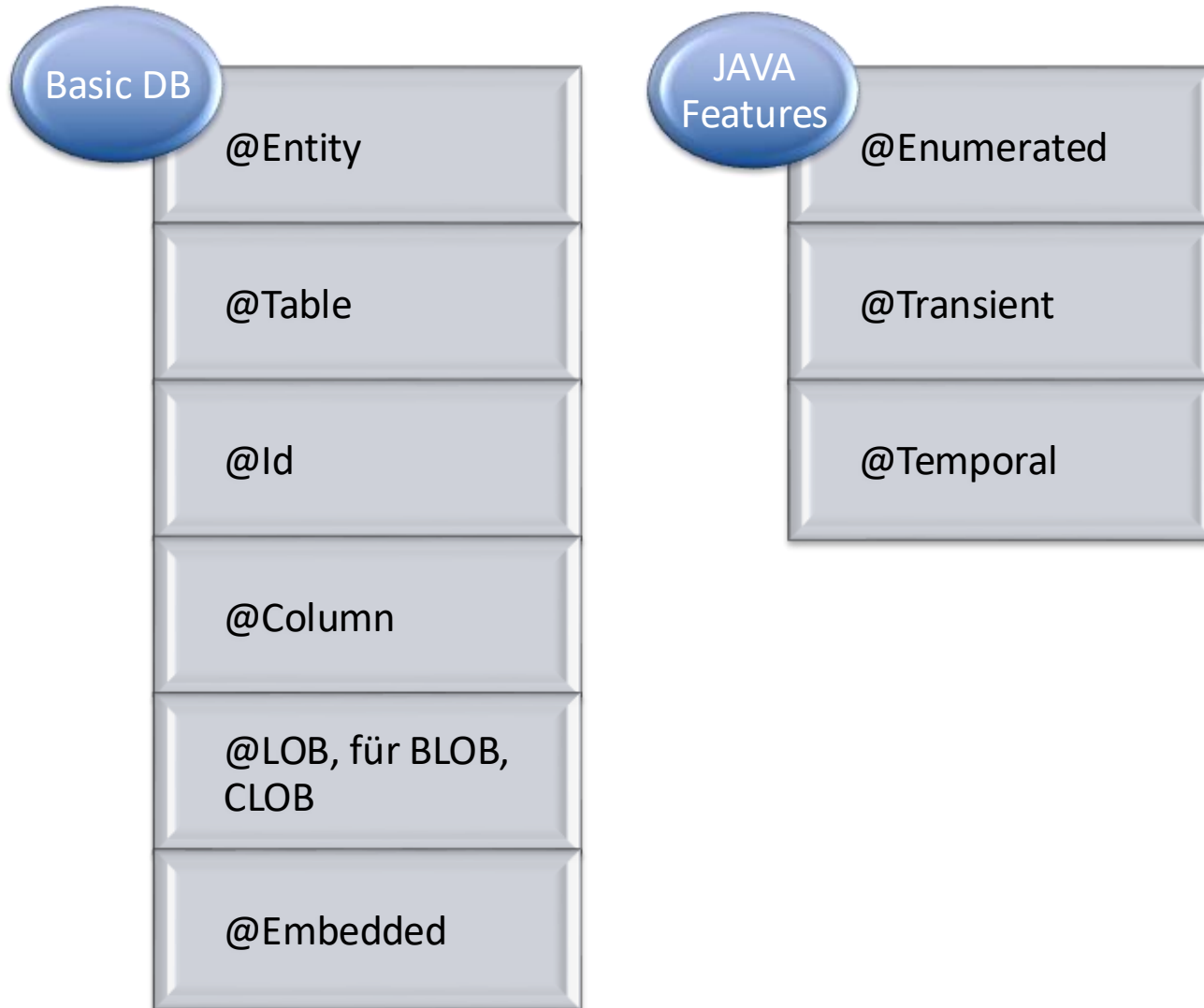
Aufgabe 1_1

- Maven Modul 1_1_JPA_DB_CONFIG zu finden
 - Konfiguration der Datei `src/META-INF/persistence.xml`.
 - Füge den `<provider>` für Hibernate hinzu
 - Konfiguriere die Datenbankverbindung mit den Infos aus der vorangegangenen Übung (url, user, password, driver class)
 - Hibernate spezifische Properties
 - `hibernate.dialect`:
`org.hibernate.dialect.DerbyDialect`
 - `hibernate.hbm2ddl.auto`: `create-drop`
 - `hibernate.show_sql`: `true`
(optional, zur Fehlersuche hilfreich)
 - Verbindung testen in `TestConnection`
- Der Test ist erfolgreich, wenn die EntityManagerFactory korrekt erstellt werden kann und `entityManager.isOpen` true returned





Basic Mapping



@Entity

```
@Entity()
```

```
public class Article
```

- `@Entity(name="ArticleEntity")`
 - Angabe des Entitynamens möglich
 - Configuration by default → unqualifizierter Klassenname = Entityname
 - Wichtig für spätere Queries mit EJBQL Generierung

@Table

- Angabe der Tabelle in der das Entity gespeichert werden soll
 - Configuration by default: Tabellenname = unqualifizierter Klassenname

```
@Table(name = "tbl_article", schema = "hr",  
       uniqueConstraints = {  
           @UniqueConstraint(columnNames = { "article_nr" }  
       )  
    })
```

Aufgabe 1_2_1

- Mappen Sie die ID der Klasse VehicleType als minimale Entity
- Testen Sie das Mapping mit der Testklasse
- Der folgende Unittest läuft grün

```
@Test
public void testFind() {
    //SELECT * from VehicleType where id= ...
    VehicleType VehicleType =
super.manager.find(VehicleType.class, id);
    assertNotNull(VehicleType);
    assertEquals(id, VehicleType.getId());
}
```

- Überprüfen Sie das erzeugte DB Schema



@Column

- Configuration per default: Spaltenname = Feldname
- Abweichend:
 - columnDefinition : String
 - name : String
 - table : String
 - insertable : boolean
 - length : int
 - precision : int
 - scale : int
 - nullable : boolean
 - unique : boolean
 - updateable : boolean

@Id

```
private String id;
```

Primary Key



Overview ID Generator Strategy



Auto-Generated IDs:

Automatically generated by the database (e.g., auto-increment).

Annotation:

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```



Table-based Generation:

Using a special table to store the current ID counter.

Annotation:

```
@GeneratedValue(strategy = GenerationType.TABLE)
```



Sequence-based Generation:

Using a database sequence to generate IDs (especially in Oracle, PostgreSQL).

Annotation:

```
@GeneratedValue(strategy = GenerationType.SEQUENCE)
```



UUID Generation:

Generate unique identifiers using UUID.

Annotation:

```
@GeneratedValue(strategy = GenerationType.AUTO)  
(often used with UUID type).
```

Example Simple IdGenerator

- `@Id`
`@GeneratedValue(strategy=GenerationType.AUTO)`
`private Long id;`

Example Sequence Generator

- `@SequenceGenerator(name = "Example_GEN", sequenceName = "DB_Example_SEQ")`
`@Id`
`@GeneratedValue(strategy = GenerationType.SEQUENCE,`
`generator = "Example_GEN")`
`private Long id;`

Aufgabe 1_2_4

- Mappen Sie die ID der Klasse VehicleType mit einem Sequenzgenerator
- Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema



Example Table Generator

- `@TableGenerator(name = "Example_GEN", table = "tbl_ExampleGen", pkColumnName = "PK_COLUMN", valueColumnName = "VALUE_COLUMN", pkColumnValue = "Example_ID", allocationSize = 10)`
`@Id`
`@GeneratedValue(strategy = GenerationType.TABLE, generator = "Example_GEN")`
`private Long id;`

`tbl_ExampleGen`

PK_COLUMN	VALUE_COLUMN
EXAMPLE_ID	12
USER_ID	12

Wer die Wahl hat, hat ...



Performance:

IDENTITY bei Batch Inserts eher schlecht.



Database Compatibility:

Nicht alle Datenbanken unterstützen alle Strategien.



Customization:

Sequence Name, InitialValue, oder Increment können über Annotations angepasst werden.

Composite Keys – Motivation & Einsatz

- In relationalen Datenbanken bestehen Primärschlüssel häufig aus **mehreren Spalten**, z. B.:
 - Bestellnummer + Positionsnummer
 - Benutzer-ID + Rollenname
 - Fahrzeug-ID + Standort + Datum
- In JPA müssen solche **zusammengesetzten Schlüssel explizit modelliert** werden. Dafür gibt es **zwei offizielle Strategien**:
 1. **@EmbeddedId** (empfohlen)
 2. **@IdClass** (Legacy / Sonderfälle)

Composite Keys in JPA – Überblick

Strategie	Kurzbeschreibung	Strategie
@EmbeddedId	Schlüssel als eigenes Value Object	@EmbeddedId
@IdClass	Schlüsselattribute direkt in der Entity	@IdClass

 **Best Practice:**

@EmbeddedId ist der empfohlene Standard

@EmbeddedId – Der Schlüssel als Value Object

- Der Primärschlüssel wird als **eigene Klasse modelliert**
- Diese Klasse ist **kein Entity**
- Sie wird vollständig in die Entity eingebettet
- Entspricht dem **OO-Value-Object-Gedanken**

@Embeddable

```
public class OrderItemId implements Serializable {  
  
    private Long orderId;  
    private Integer position;  
  
    public OrderItemId() {}  
  
    public OrderItemId(Long orderId, Integer position) {  
        this.orderId = orderId;  
        this.position = position;  
    }  
  
    @Override  
    public boolean equals(Object o) {  
        ...  
    }  
  
    @Override  
    public int hashCode() {  
        ...  
    }  
}
```

@EmbeddedId: Entity Mapping

- Kein zusätzliches Mapping nötig
- JPA erzeugt automatisch einen **Composite Primary Key**

```
@Entity
@Table(name = "ORDER_ITEM")
public class OrderItem {
```

```
    @EmbeddedId
    private OrderItemId id;
```

- SELECT oi
- FROM OrderItem oi
- WHERE oi.id.orderId = :orderId

```
    private String articleName;
    private int quantity;
}
```

@IdClass – Alternative Strategie

- Schlüsselattribute liegen **direkt in der Entity**
- Key-Klasse enthält nur **Duplikate**
- Schwächere Kapselung
- Höhere Fehleranfälligkeit
-  **Nur für Sonderfälle oder Legacy Code**

@IdClass – Beispiel

```
public class OrderItemId  
implements Serializable {  
  
    private Long orderId;  
  
    private Integer position;  
  
}
```

```
@Entity  
@IdClass(OrderItemId.class)  
public class OrderItem {
```

```
    @Id  
    private Long orderId;
```

```
    @Id  
    private Integer position;
```

```
    private String article
```

@EmbeddedId vs. @IdClass

Kriterium	EmbeddedId	IdClass
OO-Design	✅ gut	❌ schlecht
Wartbarkeit	✅ hoch	❌ niedrig
JPQL	✅ sauber	⚠️ unübersichtlich
Empfehlung	✅ Standard	❌ vermeiden

Wenn ihr euch entscheiden müsst:

👉 **EmbeddedId ist fast immer die richtige Wahl**

Typische Fehler bei Composite Keys

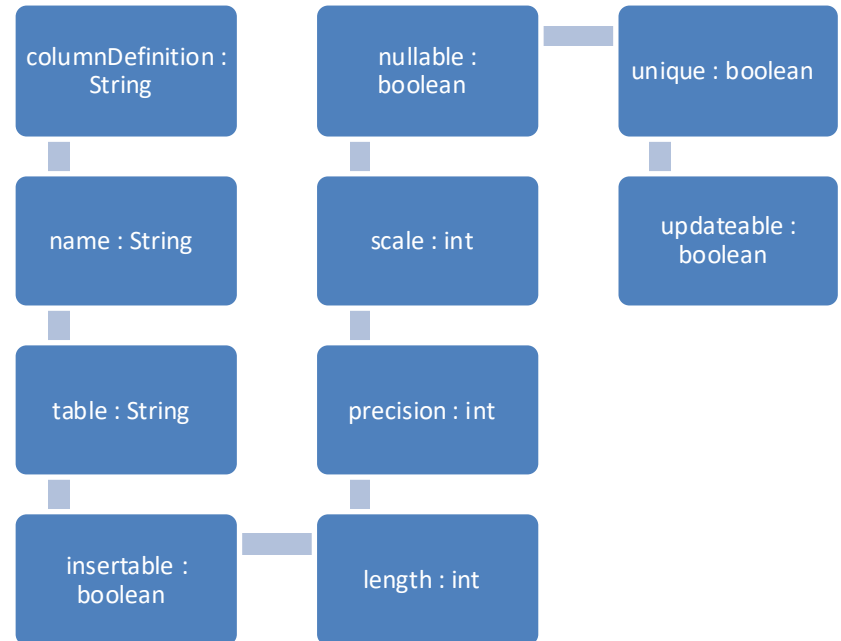
-  equals/hashCode vergessen
-  mutable Key-Klassen
-  @GeneratedValue bei Composite Keys
-  @IdClass ohne Not
-  Composite Keys sind **fachlich stabil**

Aufgabe 1_2_5:

- Nutzen Sie **Nutzer** als eine IdClass für die Entity **Kunde**
- Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema



Attribute <--
> Column



@Column

```
@Column(name="DESC", nullable=false, length=512)  
public String getDescription() { return description; }
```

```
@Column(name="DESC", columnDefinition="CLOB NOT NULL",  
table="EMP_DETAIL")
```

```
@Lob
```

```
public String getDescription() { return description; }
```

```
@Column(name="ORDER_COST", updatable=false, precision=12,  
scale=2)
```

```
public BigDecimal getCost() { return cost; }
```



Integer

Numeric



Typeconverter **by DEFAULT**

Aufgabe 1_3

- Beschreiben Sie das Mapping der Attribute der Klasse FahrzeugType wie folgt:
 - brand länge=30 Pflichtfeld
 - modell länge = 50 Pflichtfeld
 - hp vierstelliger Integer
 - maxKpH dreistelliger Integer
- Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema





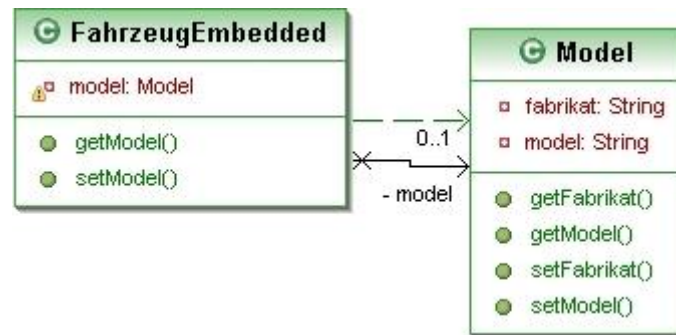
Kleines Quiz

1. Wie wird die maximale Spaltenlänge angepasst?
 - Und was passiert wenn das Attribut mit einem längeren Wert befüllt wird.
2. Welcher Datentyp braucht man javaseitig um Precision (NUMBER Datentyp) wirksam werden zu lassen.

OR Mapper Challenge

- Datentypen (Relational, OO)
- Identität – Primärschlüssel
- Mengen --> Spalten
- Kardinalität (1:n, n:m)
- Vererbung

@Embedded und @Embeddable



@Embeddable

@Embeddable

```
public class Model {  
  
    private String model;  
    private String fabrikat;
```

@Embedded

@Entity

```
public class FahrzeugEmbedded {
```

```
    @Embedded
```

```
    @AttributeOverrides( {
```

```
        @AttributeOverride(name = "model", column =  
        @Column(name = "model_model")),
```

```
        @AttributeOverride(name = "fabrikat",  
        column = @Column(name = "model_fabrikat"))  
    })
```

```
private Model model;
```

Aufgabe 1_4

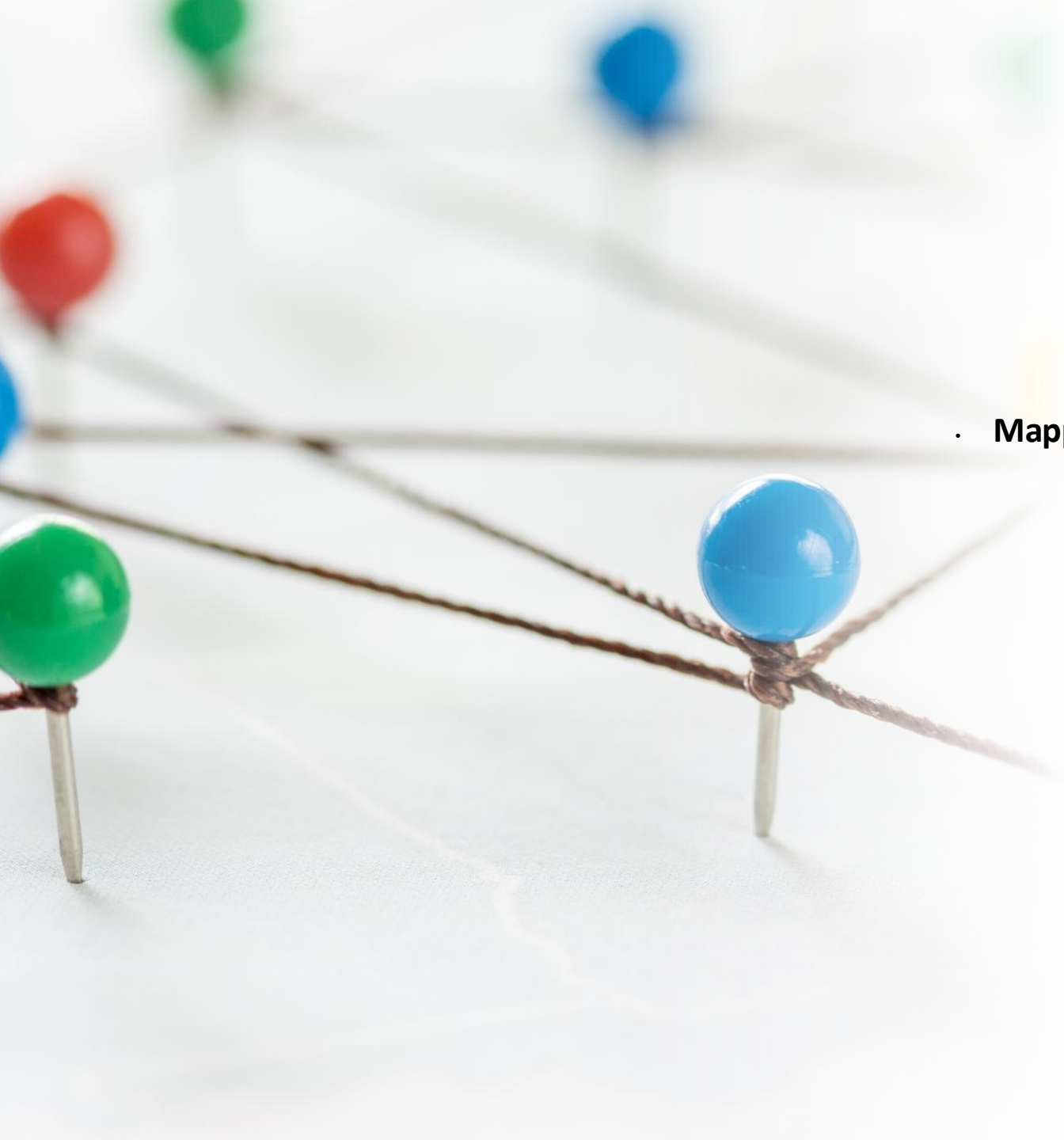
- Lagern Sie die Attribute fabrikat und modell der Klasse VehicleType in einer neuen Klasse Model ab.
- In der Datenbank sollen die beiden Attribute jedoch in der Tabelle tbl_VehicleType verbleiben
- Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema





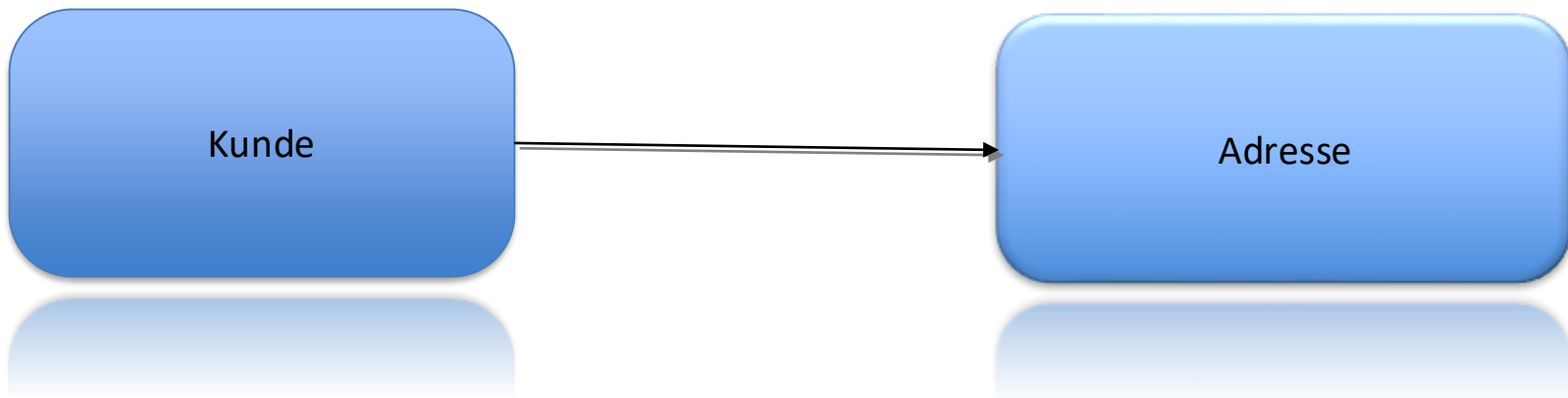
Kleines Quiz

1. Mehrstufig? Ist ein Embeddable in einem Embedded möglich?
 - Versuchen Sie es, z.B. ein `VehicleModel` als `Embedded`, welches Model enthält und z.B: noch ein Attribut *series*.
2. Wie kann ich den Spaltennamen eines embedded Attributs anpassen



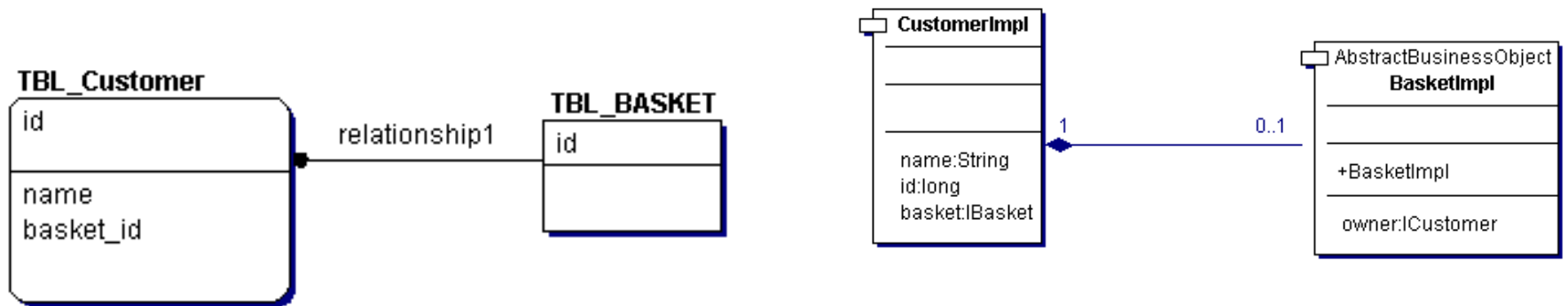
· Mapping von Beziehungen

Assoziation



ARTEN





One-To-One

```
@Entity
```

```
public class Customer {
```

```
...
```

```
@OneToOne
```

```
@JoinColumn (name="basket_id")
```

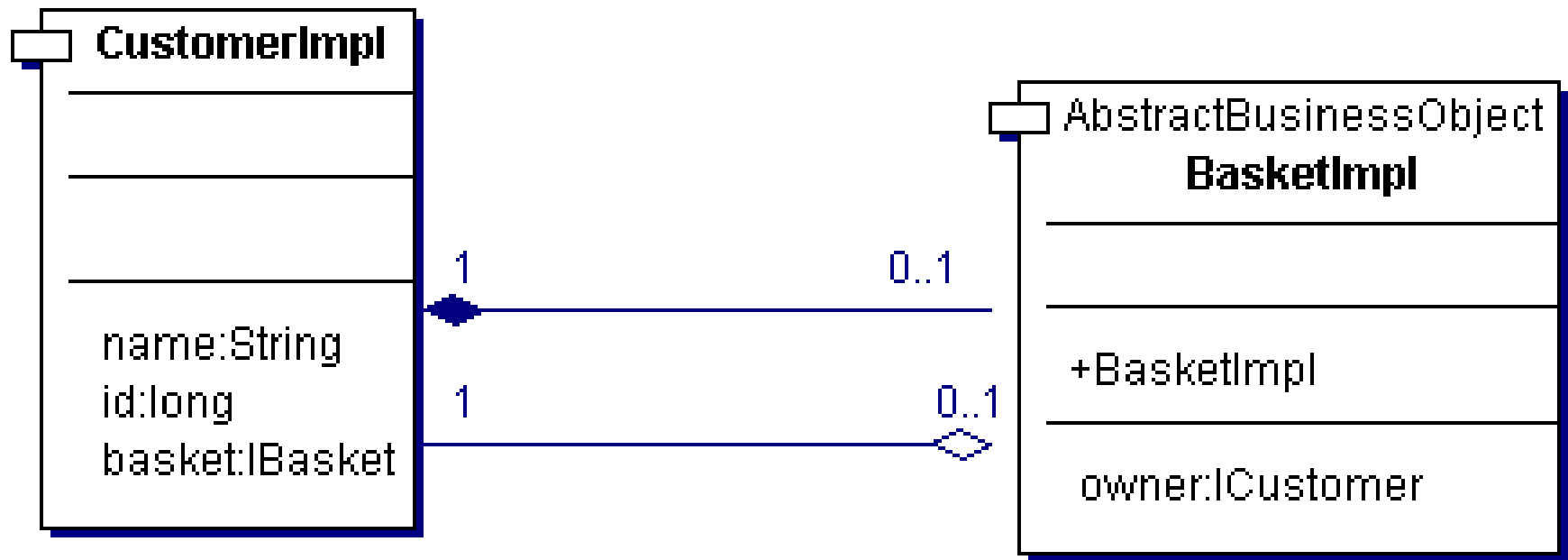
```
public Basket basket;
```

Implicit JoinColumn

@OneToOne

@PrimaryKeyJoinColumn

public Basket basket



One-To-One - Bidirectional

```

public class Basket extends AbstractBusinessObject
implements IBasket {

    @OneToOne(mappedBy="basket")
    private Customer owner;
}
  
```

Aufgabe 1_5

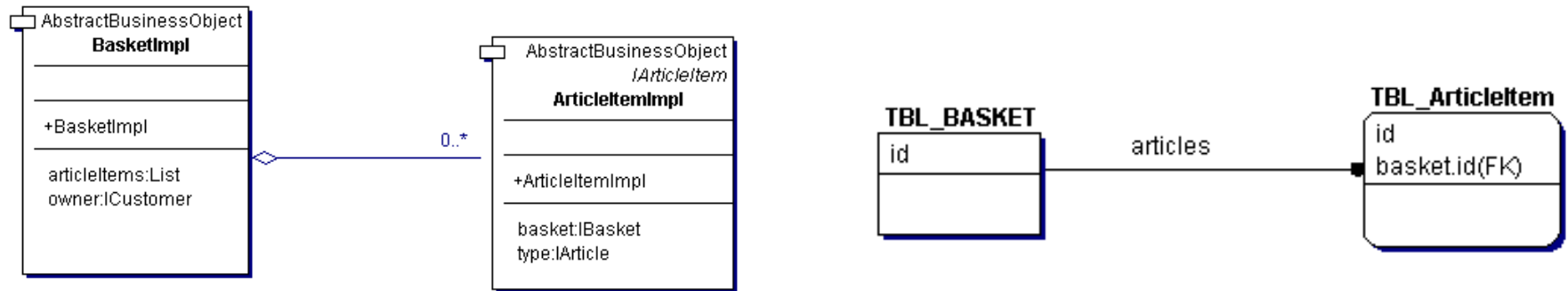
- Mappen Sie die Beziehung User
-> Person
 - Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema





Kleines Quiz

- In welcher Tabelle befindet sich die Fremdschlüsselspalte?



One-To-Many

```
@OneToMany()  
@JoinColumn(name="basket_id")  
private List<ArticleItem> getArticleItems();
```

```
@JoinTable(name="basket_article"  
    ,joinColumns={@JoinColumn(name="basket_id")}  
    ,inverseJoinColumns={@JoinColumn(name="articleItem_id")})
```

orphanRemoval=true



Constantin Film

AB 14. SEPTEMBER 2006 NUR IM KINO

CONSTANTIN FILM UND BERND EICHINGER ZEIGEN

DAS PARFUM
DIE GESCHICHTE EINES MÖRDERS

NACH DEM ROMAN VON PATRICK SÜSKIND
EIN TOM TYKWER FILM EINE BERND EICHINGER PRODUKTION

Aufgabe 1_6

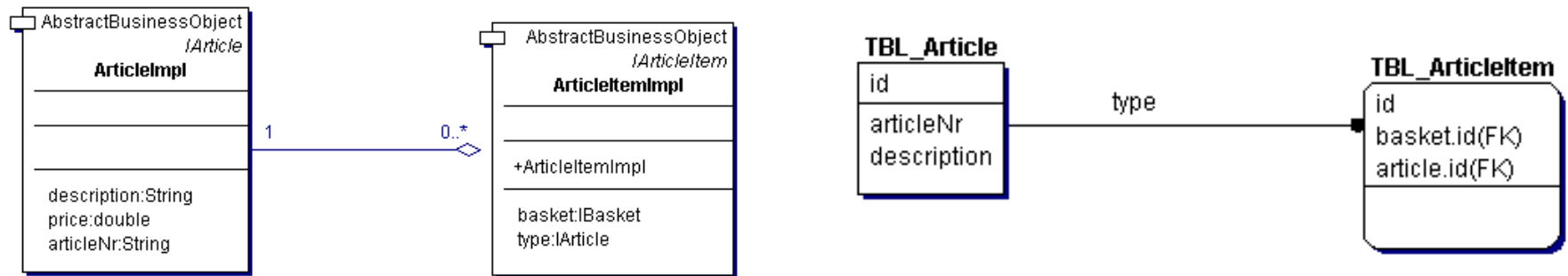
- Mappen Sie die Beziehung Shop -> Vehicle
- Mappen Sie die Vehicle Klasse
 - Die Attribute type und location sollen vom Persistenzprovider ignoriert werden
- Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema





Kleines Quiz

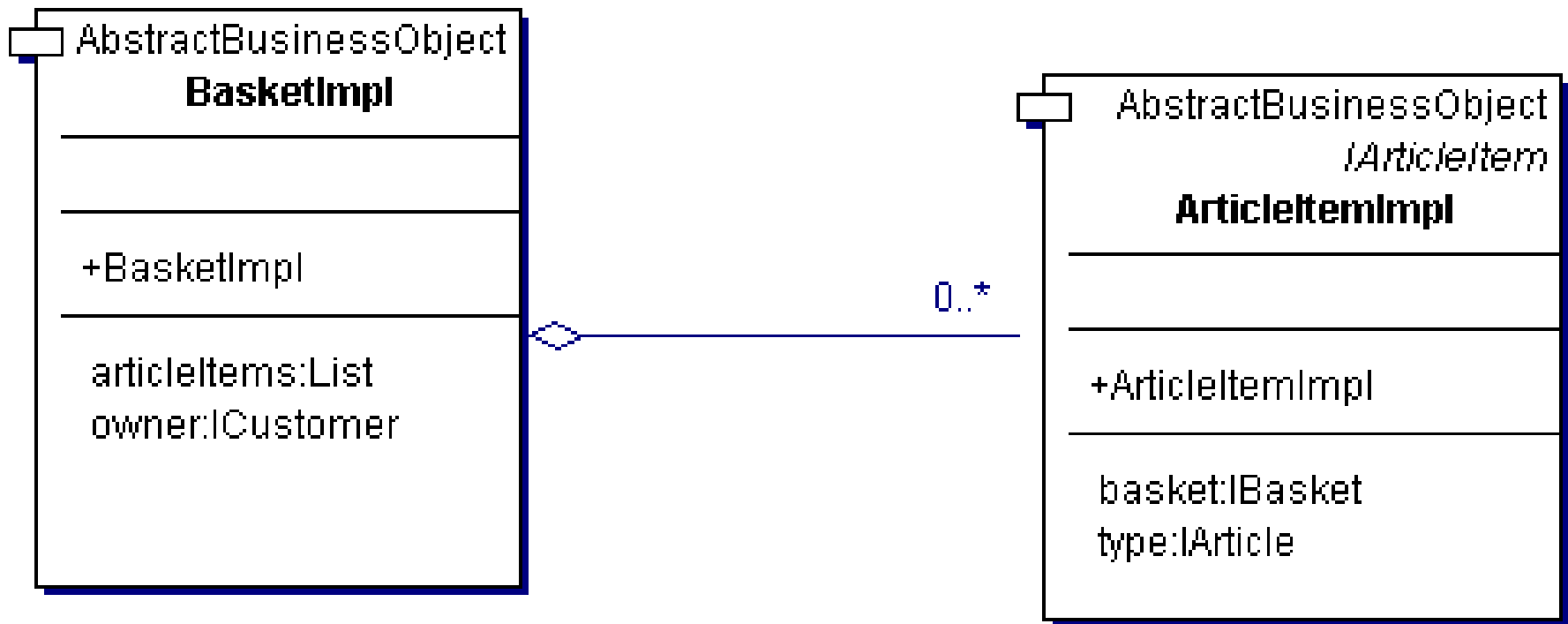
1. Welche Datentypen machen für eine 1:n Beziehung in Java nur einen Sinn?
2. Warum ist "vehicleId" kein guter Namen für die Fremdschlüsselbeziehung der in der Übung abgebildeten Beziehung



Many-To-One

Many-To-One

```
@ManyToOne  
@JoinColumn(name="article_id")  
Private Article type;
```



One-To-Many - Bidirectional

```
@OneToMany(mappedBy="basket")  
private List<ArticleItem> getArticleItems();
```

Aufgabe 1_7

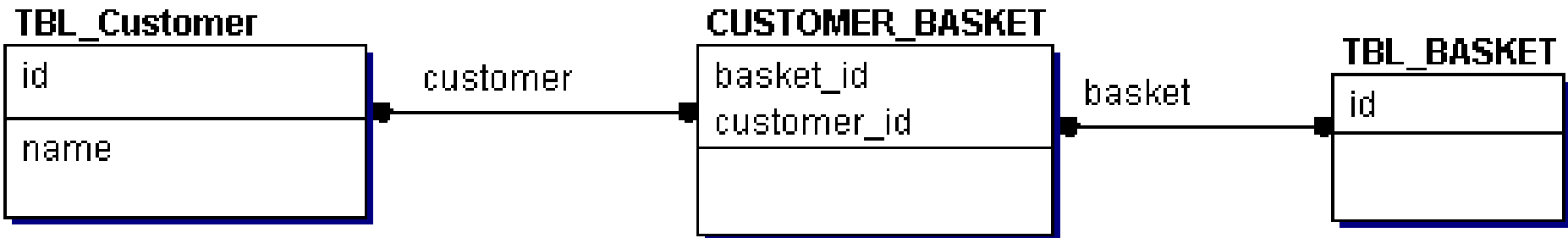
- Mappen Sie die folgenden Attribute der Klasse Vehicle
 - location (Achtung bidirektional)
 - type (Viele Fahrzeuge haben den gleichen Typ)
- Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema





Kleines Quiz

1. Was müssten wir in VehicleType ergänzen, um die Rückrichtung der Beziehung zu modellieren.



Many-To-
Many

```
@ManyToMany
```

```
@JoinTable(name="CUSTOMER_BASKET"
    ,joinColumns={@JoinColumn(name="basket_id")}
    ,inverseJoinColumns={@JoinColumn(name="customer_id")})
private List<Customer> owners;
```

<i>Collection Typ</i>	<i>Annotationen</i>	<i>Inhalt</i>	<i>Beschreibung</i>
java.util.List	@OrderBy	(<i>Spalte</i> ASC/DESC)	Reihenfolge auch in DB
java.util.Map	@MapKey	(name=“ <i>Spalte</i> “)	Keine definierte Reihenfolge hashCode und equals für Schlüssel und Werte Comparator

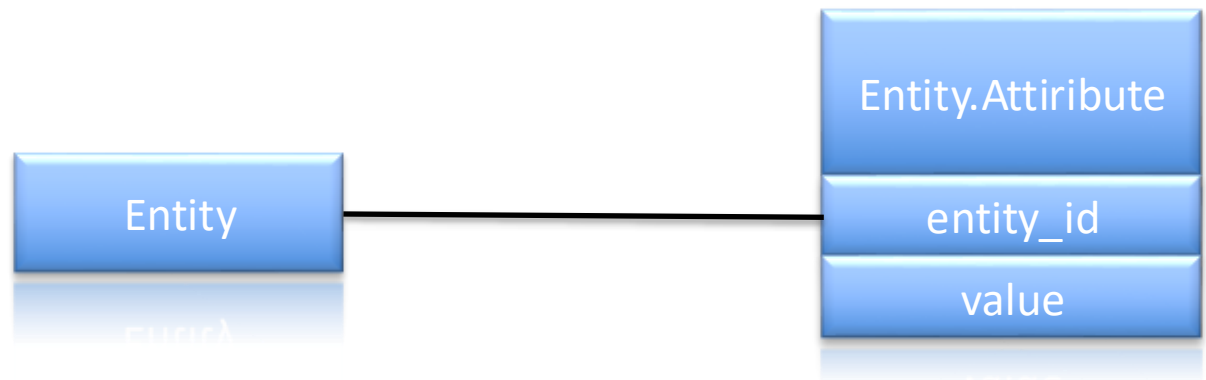
Collection Typen

A black and white photograph showing a perspective view down a long factory floor. Numerous workers, mostly women, are seated at rows of sewing machines. They are all focused on their work. The machines are industrial-grade, with visible spools of thread and bobbins. The workers are wearing light-colored, short-sleeved shirts. The floor is dark and polished. The overall atmosphere is one of intense, organized labor.

Noch mehr List..

Auch für Map

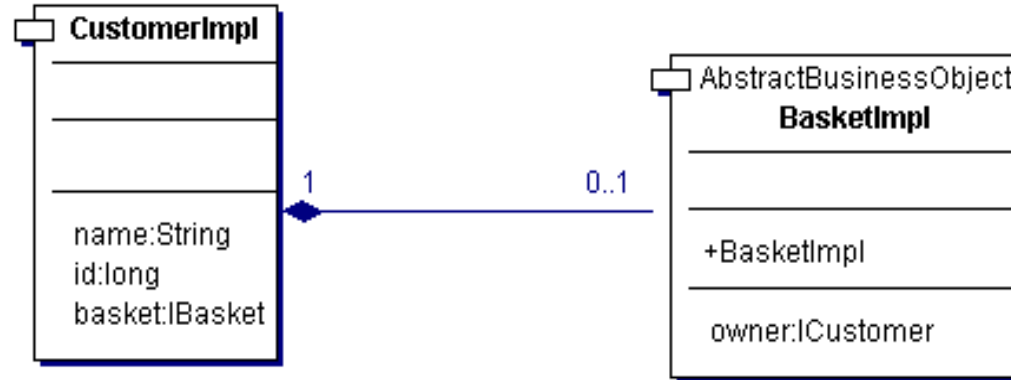
```
@CollectionTable( name = "entity.attribute")  
@ElementCollection  
private Set<String> someStringColl = new  
HashSet<String>();
```





@OrderColumn

Cascading



Cascading

- In allen Beziehungstypen
 - Attribute cascade={ Typen }
 - Enumeration CascadeType
 - `@OneToOne(cascade = CascadeType.PERSIST)`
- Reduziert EntityManager Aufrufe
- **Performance killer!!!**
 - Anzahl an DB- Anfragen

CascadeType.*

CascadeType.PERSIST

CascadeType.MERGE

CascadeType.REMOVE

CascadeType.REFRESH

CascadeType.ALL

Aufgabe 1_8

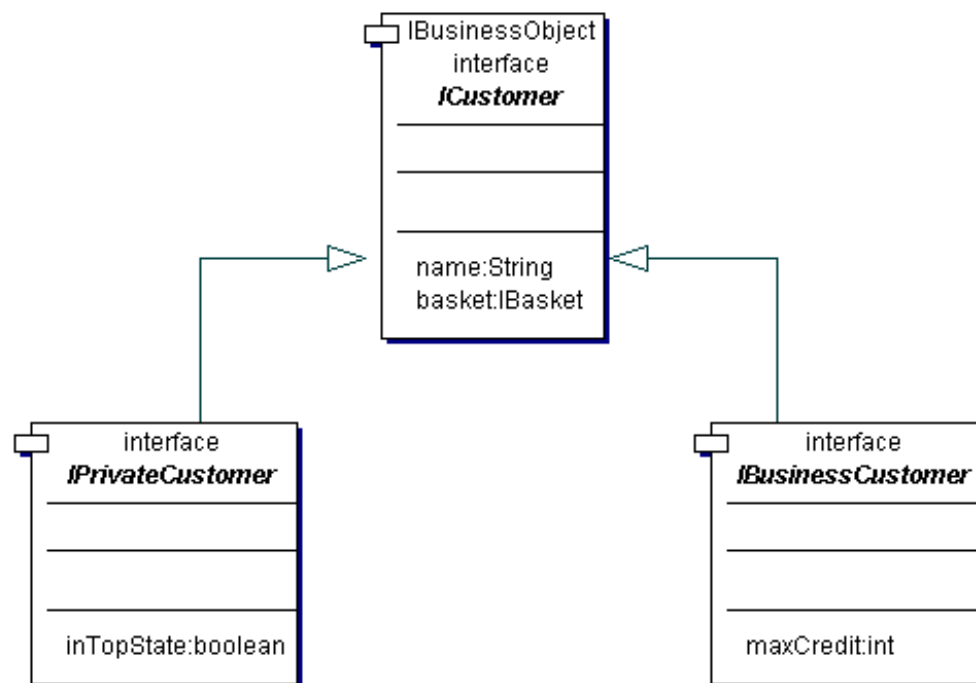
- Erweitern Sie die Klasse Vehicle um ein Attribut:
 - `List<Shop> locationHistory`
 - Dies solle alle Ehemaligen Standorte beinhalten
 - Die Funktion `setLocation` soll den zu ersetzenden Standort in die Historie eintragen.
 - Mappen Sie das neue Attribut auf die DB
- Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema





Kleines Quiz

1. Die Beziehung wird durch eine Zwischentabelle abgebildet. Könnte man die Zwischentabelle als JPA-Entität abbilden? Wie sähe diese Klasse aus?



Strategien



TBL_CUSTOMER_HIERARCHY

id
name basket_id inTopState maxCredit DISCRIMINATOR

```
@Entity
@Table (name="TBL_CUSTOMER_HIERARCHY")
@Inheritance (strategy=
    InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn (name="DISCRIMINATOR"
    ,discriminatorType=DiscriminatorType.STRING)
@DiscriminatorValue ("thevalue")
```



Vorteile

Nachteile

- Einfach
- Performance
 - JOINS/UNIONS
- Keine NOT NULL Spalte (Unterklassen)
- Keine Normalisierung

Aufgabe 1_9_1

- Mappen Sie die Hierarchie VehicleType, Car, Truck mittels SingleTable
 - Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema

```
@Test
public void testInheritance() {
    assertNotNull(super.manager.find(Truck.class,
    Truck.getId()));
}
```





Kleines Quiz

1. Wie sieht der Inhalt der Discriminator Spalte aus, wenn sie die Daten vom Testdurchlauf commiten?

Table per Concrete Class

TBL_Customer

id
name basket_id

TBL_PrivateCustomer

id
name basket_id inTopState

TBL_BusinessCustomer

id
name basket_id maxCredit

```
@Entity
@Table(name="TBL_Customer")
@Inheritance(strategy=InheritanceType.TABLE_PER_CLASS)
```

```
@Entity
@Table(name="TBL_PrivateCustomer")
```

```
@Entity
@Table(name="TBL_BusinessCustomer")
```

- Constraints

- Keine Normalisierung
- Multiple Anfragen oder Unions beim Laden

A close-up photograph of two human fists bumping in the center of the frame. The fist on the left is slightly larger and has more visible hair on the forearm. The fist on the right is slightly smaller. The background is plain white.

Vorteile

Nachteile

Aufgabe 1_9_2

- Mappen Sie die Hierarchie VehicleType, Car, Truck mittels TablePerClass
 - Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema

```
@Test
public void testInheritance() {

    assertNotNull(super.manager.find(Truck.class,
    Truck.getId()));
}
```





Kleines Quiz

1. Markieren Sie die Klasse `VehicleType` als abstrakt. Welche Änderungen ergeben sich im Schema?

Table per Subclass

TBL_Customer

id
name basket_id

TBL_PrivateCustomer

id
inTopState

TBL_BusinessCustomer

id
maxCredit

```
@Entity
@Table(name="TBL_Customer")
@Inheritance(strategy=InheritanceType.JOINED)
```

```
@Entity
@Table(name="TBL_PrivateCustomer")
```

```
@Entity
@Table(name="TBL_BusinessCustomer")
```

Aufgabe 1_9_3

- Mappen Sie die Hierarchie VehicleType, Car, Truck mittels Joinedao
 - Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema

```
@Test
public void testInheritance() {
    assertNotNull(super.manager.find(Truck.class,
    Truck.getId()));
}
```





Kleines Quiz

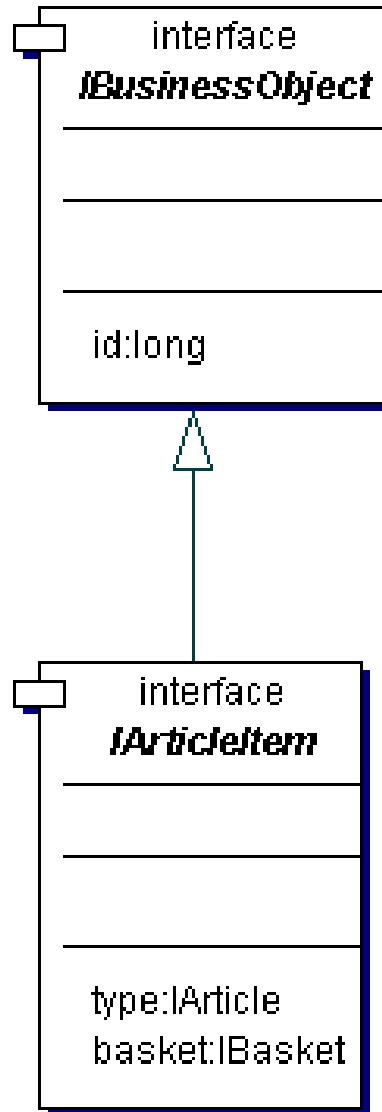
1. Markieren Sie die Klasse `VehicleType` als abstrakt. Welche Änderungen ergeben sich im Schema?

Nachteile

- Performance
- Single Table

Vorteile

- Normalisierung
- Constraints



Basisklassen

(Abstract) Superentity

Basisklasse

```
@MappedSuperclass
```

Unterklassen

```
@Entity
```

```
@PrimaryKeyJoinColumn (name="NEW_PK")
```

```
@AttributeOverride (name="alterName"  
    column=@Column (name="neuerName") )
```

Aufgabe 1_9_4: Teil 1

- Basisklasse erstellen: Erstelle im Paket `net.rentacar.model` eine neue abstrakte Klasse mit dem Namen `AbstractBusinessObject`.
 - Annotiere die Klasse mit `@MappedSuperclass`.
 - Verschiebe das Primärschlüssel-Feld `id` (String) aus den bestehenden Entitäten in diese Basisklasse.
 - Füge ein neues Feld `version` vom Typ `long` hinzu. Annotiere es mit `@Version`, damit JPA dieses Feld für das Optimistic Locking verwendet. Optional: Gib der Spalte über `@Column(name="optimisticLocking")` einen spezifischen Namen.
 - Logik zentralisieren: Implementiere in `AbstractBusinessObject` einen Standardkonstruktor, der automatisch eine neue UUID als ID generiert, sofern keine übergeben wurde.



Aufgabe 1_9_4: Teil 2

- Entitäten anpassen:
 - Lasse alle relevanten Entitäten (z. B. Vehicle, Person, Shop, VehicleType, Model, Nutzer) von AbstractBusinessObject erben.
 - Entferne die nun redundanten id-Felder und deren Getter aus diesen Entitäten.
 - Passe die Konstruktoren der Entitäten so unter Verwendung von super(id) an, dass sie die ID-Initialisierung an die Basisklasse delegieren.



Aufgabe 1_9_4: Erfolgskriterien

- Acceptance Criteria
 - Das Projekt lässt sich fehlerfrei kompilieren.
 - In der Datenbank erhalten alle Tabellen der betroffenen Entitäten automatisch eine zusätzliche Spalte für die Versionierung (z. B. optimisticLocking).
 - Beim Erzeugen neuer Instanzen wird automatisch eine UUID generiert, ohne dass dies in jeder Entität einzeln implementiert werden muss.





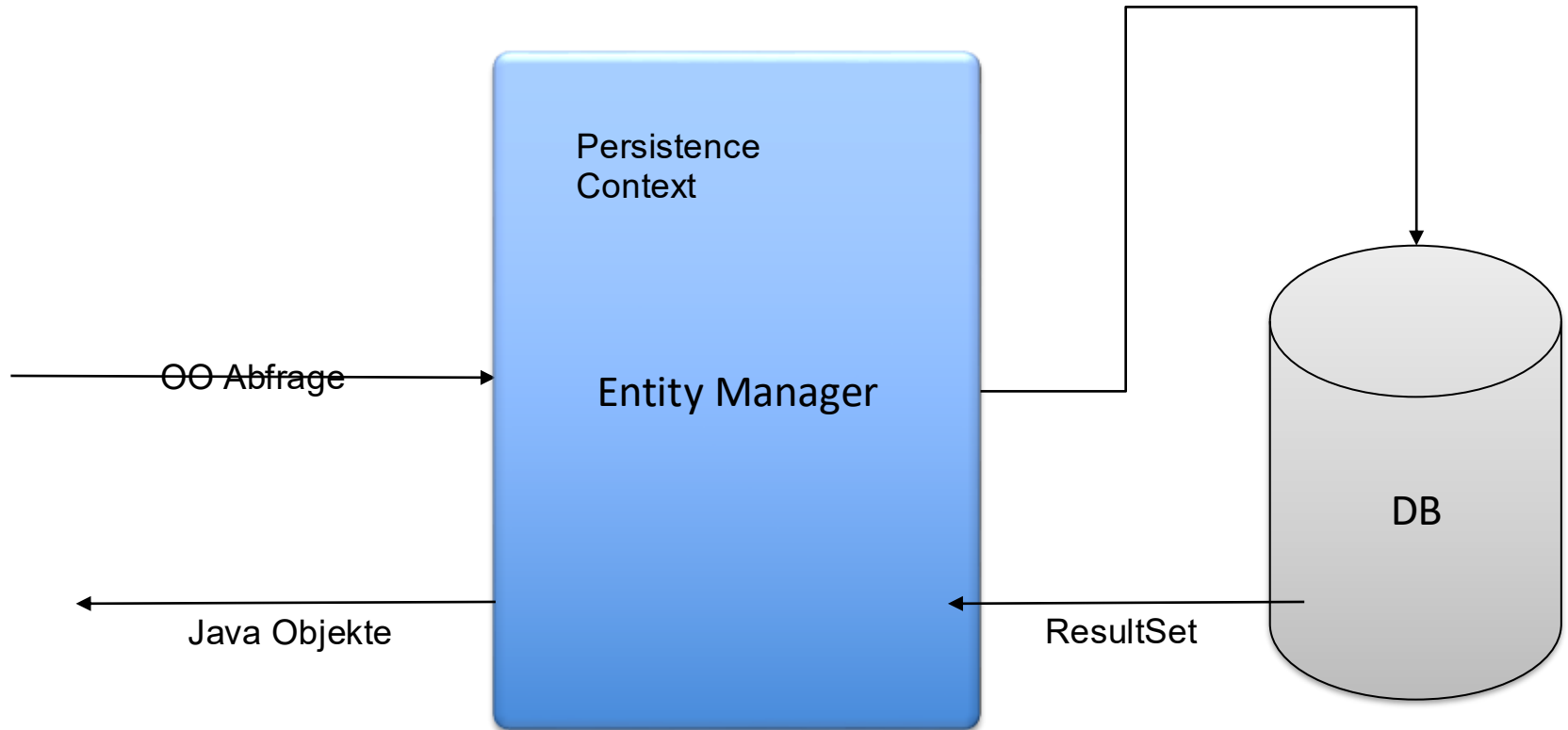
Kleines Quiz

- Abgrenzung zu Vererbungsstrategien
 - Was ist der Unterschied zu @Inheritance (SINGLE_TABLE, JOINED, TABLE_PER_CLASS)?

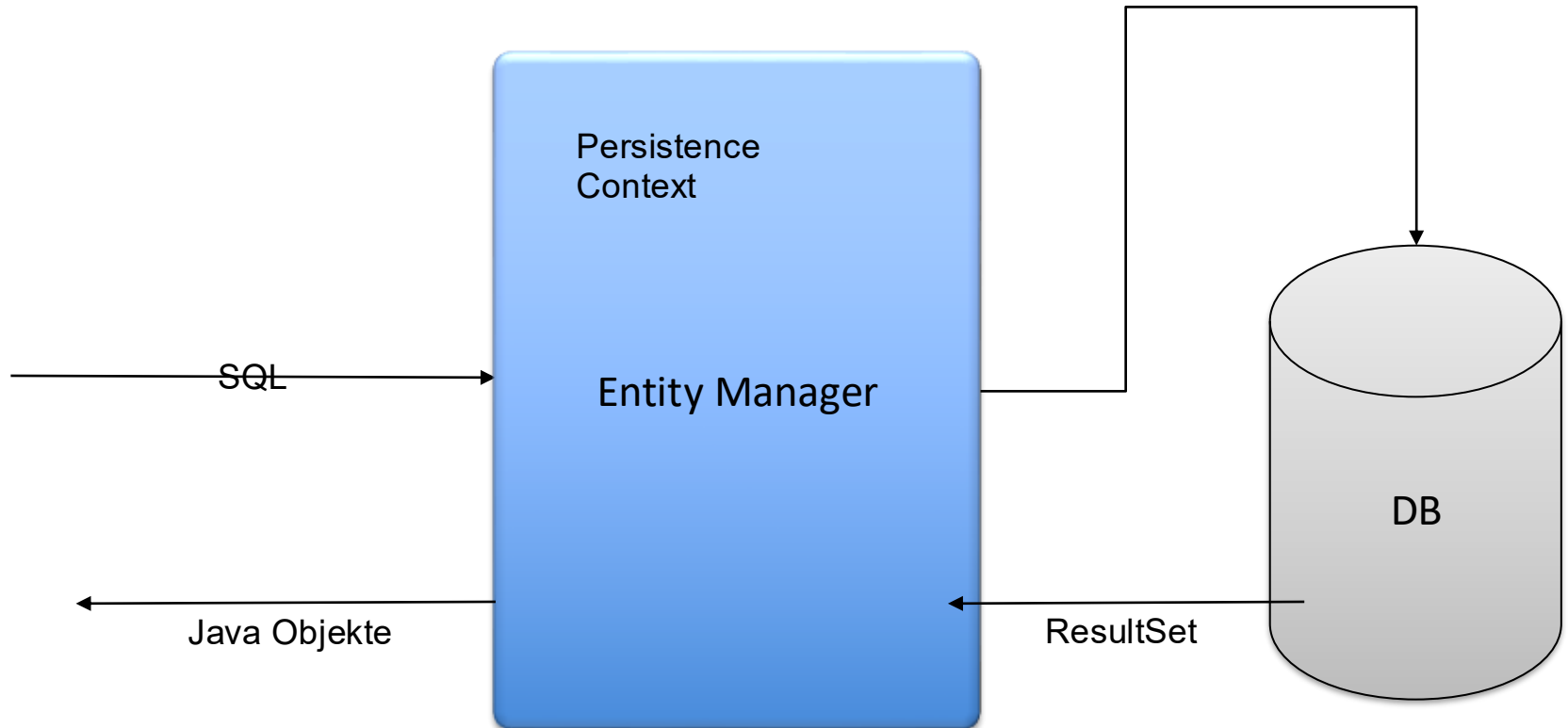


Q&A

Ablauf einer Query



Ablauf einer Native Query



Query API

EM = Factory

- TypedQuery createQuery(String queryString, Class entityClass)
- TypedQuery createNamedQuery(String name, Class entityClass)
- Query createNativeQuery(String sqlString)
- Query createNativeQuery(String sqlString, String resultSetMapping)
- Query createNativeQuery(String sqlString, Class resultClass)

Entity Manager Query API

Bezogen auf Name

- `setParameter(String name, Object value)`
- `setParameter(String name, Date value, TemporalType temporalType)`
- `setParameter(String name, Calendar value, TemporalType temporalType)`

Bezogen auf Position

- `setParameter(int position, Object value)`
- `setParameter(int position, Date value, TemporalType temporalType)`
- `setParameter(int position, Calendar value, TemporalType temporalType)`

Query API

- execute

- Resultat
 - List getResultList()
 - Object getSingleResult()
 - int executeUpdate()

Query Erzeugen

```
Query query = entityManager
    .createQuery("SELECT cust FROM Customer cust WHERE
cust.id=" + id);
```

```
Query query = entityManager
    .createQuery("SELECT bask FROM Basket bask WHERE " +
        "bask.owner.id=" + id);
```

```
Query query = entityManager
    .createQuery("SELECT bask, cust FROM Basket bask,
Customer cust");
```

```
List tuple = query.getResultList();
IBasket basket = (IBasket) ((Object[])tuple.get(0))[0];
ICustomer cust = (ICustomer) ((Object[])tuple.get(0))[1];
```

Aufgabe 2_01: Teil 1

- Wir möchten nun gezielt nach Fahrzeugtypen suchen, die eine gewisse Leistung erbringen. Deine Aufgabe ist es, eine Abfrage zu formulieren, die alle Fahrzeugtypen filtert, die mehr als 130 PS haben.
- **Aufgabe**
 - Testklasse `net.rentacar.TestQuery`
 - JPQL-Abfrage implementieren: Erweitere die Methode `testQueryForVehicleMoreThan130HP()` (oder implementiere sie), um eine Abfrage mit dem `EntityManager` auszuführen.
 - Nutze `manager.createQuery(String jpql)`.
 - Die Abfrage soll alle Instanzen von `VehicleType` finden, deren Attribut `hp` (Horsepower) größer als 130 ist.
 - Verwende `getResultList()`, um die Liste der Ergebnisse zu erhalten.



Aufgabe 2_01: Teil 2

```
@Test
public void
testQueryForVehicleMoreThan130HP() {
    // 1. JPQL String definieren
    String jpql = "SELECT ... FROM ... WHERE
    ...";

    // 2. Abfrage ausführen
    List<VehicleType> results =
manager.createQuery(jpql).getResultList();

    // 3. Verifizieren
    assertEquals(1, results.size());
}
```





Kleines Quiz

1. Was passiert, wenn du in deiner JPQL-Abfrage den Tabellennamen aus der Datenbank (z. B. `tbl_VehicleType`) anstatt des Klassennamens (`VehicleType`) verwendest?
2. Warum ist in JPQL ein Alias (z. B. `SELECT v FROM VehicleType v`) im Gegensatz zu Standard-SQL fast immer verpflichtend?
3. Welchen Datentyp haben die Elemente in der Liste, die `getResultList()` zurückgibt, wenn du `SELECT v FROM VehicleType v` ausführst? Werden hier echte Car- oder Truck-Objekte zurückgegeben, wenn diese in der Datenbank vorhanden sind?

Seitenverfahren (Paging)

- Query `setMaxResults(int maxResults)`
- Query `setFirstIndex(int firstIndex)`

```
Query query = entityManager
    .createQuery("SELECT cust FROM Customer cust");
List result = query
    .setMaxResults(max)
    .setFirstResult(index)
    .getResultList();
```

WHERE

Operatoren & Wildcards

- =, <>, >, <, >=, <=
- like
- (NOT) BETWEEN
- (NOT) IN, (NOT) MEMBER OF
- IS (NOT) EMPTY, IS (NOT) NULL

```
Query query = entityManager.createQuery("SELECT cust FROM  
Customer" + "cust WHERE cust.id = ?1");  
query.setParameter(1,id);
```

```
Query query = entityManager.createQuery("SELECT bask FROM  
Basket " + "basket WHERE basket.owner = :vergleich");  
query.setParameter("vergleich",customer);
```

Aufgabe 2_02

- Wir möchten nun gezielt nach Fahrzeugtypen suchen, die eine gewisse Leistung erbringen. Deine Aufgabe ist es, eine Abfrage zu formulieren, die alle Fahrzeugtypen filtert, die mehr als 130 PS haben.
- **Aufgabe**
 - Wie 2_01 nur mittels Nutzung eines NamedParameter für hp





Kleines Quiz

1. Wie müsste die Abfrage lauten, wenn du nicht nur nach der Leistung ($hp > 130$), sondern zusätzlich nur nach Fahrzeugtypen suchen möchtest, deren $maxKpH$ über 200 liegt?
2. Was passiert wohl, wenn man das `SELECT v` durch `SELECT v.hp` ersetzt? Was wäre dann der Rückgabetyt der Liste?

JPA QL

Sonstiges

```
Query query = entityManager
    .createQuery("SELECT cust FROM Customer cust");
query =
    query.setHints("org.hibernate.timeout", 200
    );
List result = query
    .setFlushMode(FlushModeType.COMMIT)
    .getResultList();
```

Einfache Anfragen

```
Query query = entityManager.createQuery("SELECT cust.name"  
    + " FROM Customer cust");
```

```
Query query = entityManager.createQuery("SELECT c.name,"  
    + " c.id FROM Customer c");
```

```
Query query = entityManager.  
    createQuery("SELECT new de.test.IdNameWrapper(c.name, c.id) " +  
        "FROM Customer c");
```

```
Query query = entityManager.  
    createQuery("SELECT c.basket FROM Customer c");
```

INNER JOINS

```
Query query = entityManager
    .createQuery("SELECT item FROM Basket AS b, "
        + " IN (b.articleItems) item WHERE item.name = ?1");
```

```
Query query = entityManager
    .createQuery("SELECT items FROM Basket AS b "
        + " INNER JOIN b.articleItems items");
```

```
Query query = entityManager
    .createQuery("SELECT items FROM Customer AS c, "
        + " IN (c.baskets) b, IN (b.articleItems) items");
```

LEFT (OUTER) JOINS

```
Query query = entityManager  
    .createQuery("SELECT c.name, b.id FROM Customer c  
    LEFT JOIN c.baskets b");
```

```
Query query = entityManager  
    .createQuery("SELECT c.name, b.id FROM Customer  
    c LEFT OUTER JOIN c.baskets b");
```

Aufgabe 2_04: Teil 1

- Unser Fuhrpark-System wird komplexer. Wir müssen nun Abfragen erstellen, die Nutzergruppen filtern und den Status unserer Shops (Fuhrpark-Belegung) prüfen.
- **Aufgabe** Testklasse TestQuery
 - Filtern mit IN: Implementiere die Methode `testQueryForNutzerWithPersonnameISMichaelOrMathias()`.
 - Suche nach allen User-Objekten, deren zugehörige Person einen Vornamen hat, der in der Liste ['Michael', 'Mathias'] enthalten ist.
 - Nutze einen benannten Parameter (z. B. `:names`), um die Liste zu übergeben.



Aufgabe 2_04: Teil 2

- Prüfen auf leere Listen mit IS EMPTY: Implementiere die Methode `testQueryForShopMitKeinemVehicle()`.
 - Finde alle Shop-Objekte, die derzeit keine Fahrzeuge (carpool) besitzen.
 - Nutze dafür den IS EMPTY-Operator auf dem entsprechenden Collection-Attribut der Shop-Klasse.
- Negierung mit NOT IN: Implementiere die Methode `testQueryForShopsichtInStuttgartUndMuenchen_MitNOoperator()`.
 - Finde alle Shop-Objekte, deren Standort (location) nicht in der Liste ['Muenchen', 'Stuttgart'] liegt.
 - Verwende hierfür die Kombination NOT IN.



Aufgabe 2_04: Erfolgskriterien

- Alle drei Testmethoden in TestQuery laufen grün.
- Du verwendest Parameter-Binding (setParameter) für die Listen-Werte bei den IN-Abfragen.
- Bei der IS EMPTY-Abfrage wird kein manueller Join verwendet





Kleines Quiz

1. Warum verwendet man bei der Prüfung auf eine leere Fahrzeugliste (carpool) in der Entität Shop den Operator IS EMPTY und nicht IS NULL (also WHERE s.carpool IS NULL)?
2. Stell dir vor, du hast eine Liste mit 5.000 Vornamen, die du dem IN-Operator als Parameter übergibst. Warum könnte dies problematisch sein, und was wäre eine bessere Alternative in der Datenbank-Programmierung?

Aufgabe 2_06: Teil 1

- Wir möchten herausfinden, welche unserer Shops derzeit Fahrzeuge einer bestimmten Marke im Fuhrpark haben. Da die Beziehung von Shop zu Vehicle (carpool) eine Liste ist, müssen wir einen Join verwenden, um in dieser Liste suchen zu können.
- **Aufgabe** Testklasse TestQuery
 - testInnerJoin_SearchShopWithVehicleOfBrandVW().
 - Schreibe eine JPQL-Abfrage, die alle Shop-Objekte findet, die mindestens ein Fahrzeug in ihrem carpool haben, dessen Marke (brand) den Wert 'VW' hat.
 - Verwende die Syntax INNER JOIN f.carpool fz.
 - Nutze getSingleResult(), um das Ergebnis abzurufen, und verifiziere mit assertEquals, dass der Standort des gefundenen Shops 'Stuttgart' ist.



Aufgabe 2_06: Teil 2

- Suche mit der IN-Syntax: Implementiere die Methode `testIn_SearchShopWithVehicleOfBrandVW()`.
 - Löse die gleiche Aufgabe wie oben, aber verwende diesmal die alternative JPQL-Syntax für Joins über Collections: `FROM Shop f, IN(f.carpool) fz`.
 - Verifiziere auch hier, dass der korrekte Shop gefunden wird.



Aufgabe 2_06: Erfolgskriterien

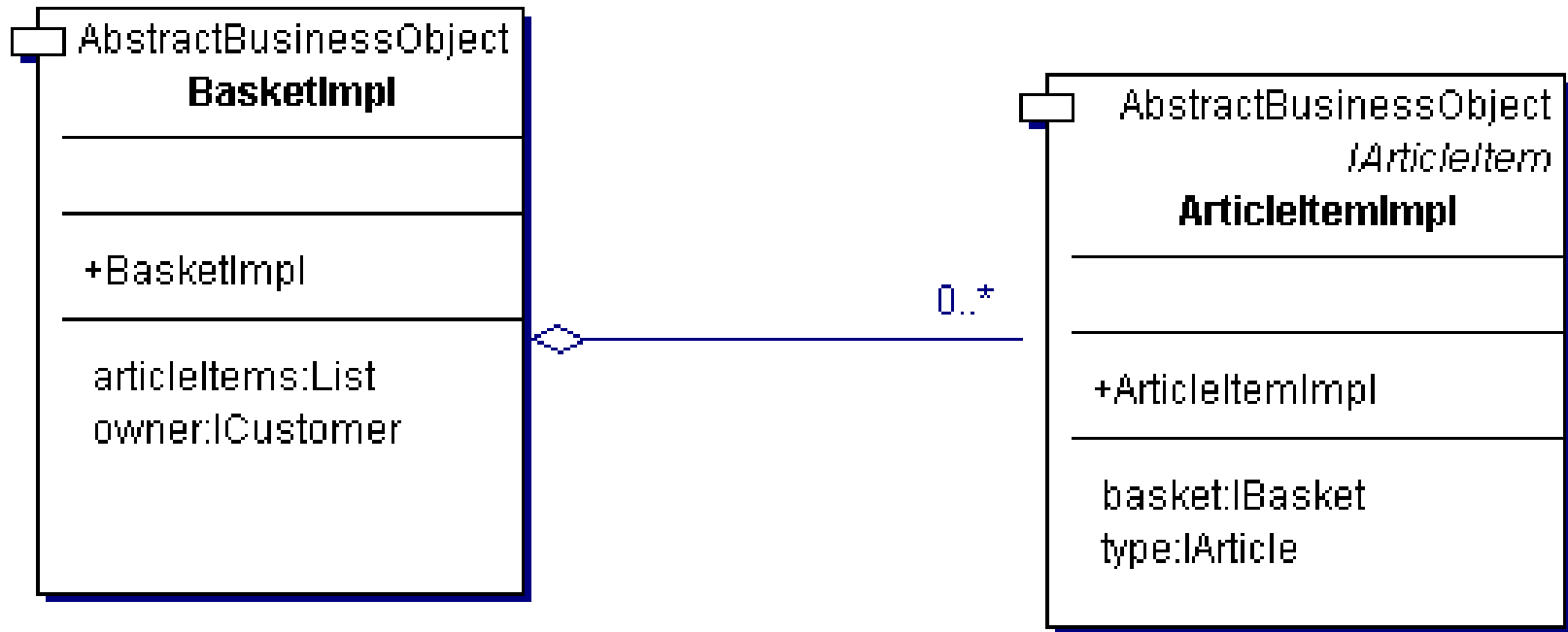
- Beide Testmethoden laufen grün.
- Du hast verstanden, wie man über eine Collection navigiert, um Attribute der darin enthaltenen Objekte zu filtern.
- Du kannst den Unterschied zwischen der Navigation über ein Einzelobjekt (f.location) und über eine Collection (JOIN f.carpool) erklären.
- Wichtiger Hinweis: In dieser Übung erwarten wir genau ein Ergebnis. Wenn eine Abfrage mehrere Shops finden würde, würde getResult() eine Exception werfen. In der Realität würde man hier eher getResultList() verwenden. Für den Test nutzen wir hier die Annahme, dass unser Test-Setup nur einen passenden Shop bereithält.





Kleines Quiz

1. Warum funktioniert die Abfrage `SELECT s FROM Shop s WHERE s.carpool.type.model.brand = 'VW'` nicht, während `SELECT v FROM Vehicle v WHERE v.type.model.brand = 'VW'` problemlos funktioniert?
2. Stell dir vor, du hast einen Shop in "Berlin" angelegt, der aber noch gar keine Fahrzeuge in seinem carpool hat. Erscheint dieser Shop im Ergebnis der Abfrage `SELECT s FROM Shop s INNER JOIN s.carpool fz`, wenn man die WHERE-Klausel weglässt?



```

List<IBasket> baskets = entityManager
    .createQuery("FROM Basket b").getResultList();

```

Performance
Probleme?

```

for (IBasket basket : baskets)
{
    for (IArticleItem item :
        basket.getArticleItems()) {
        item.getId();
    }
}

```

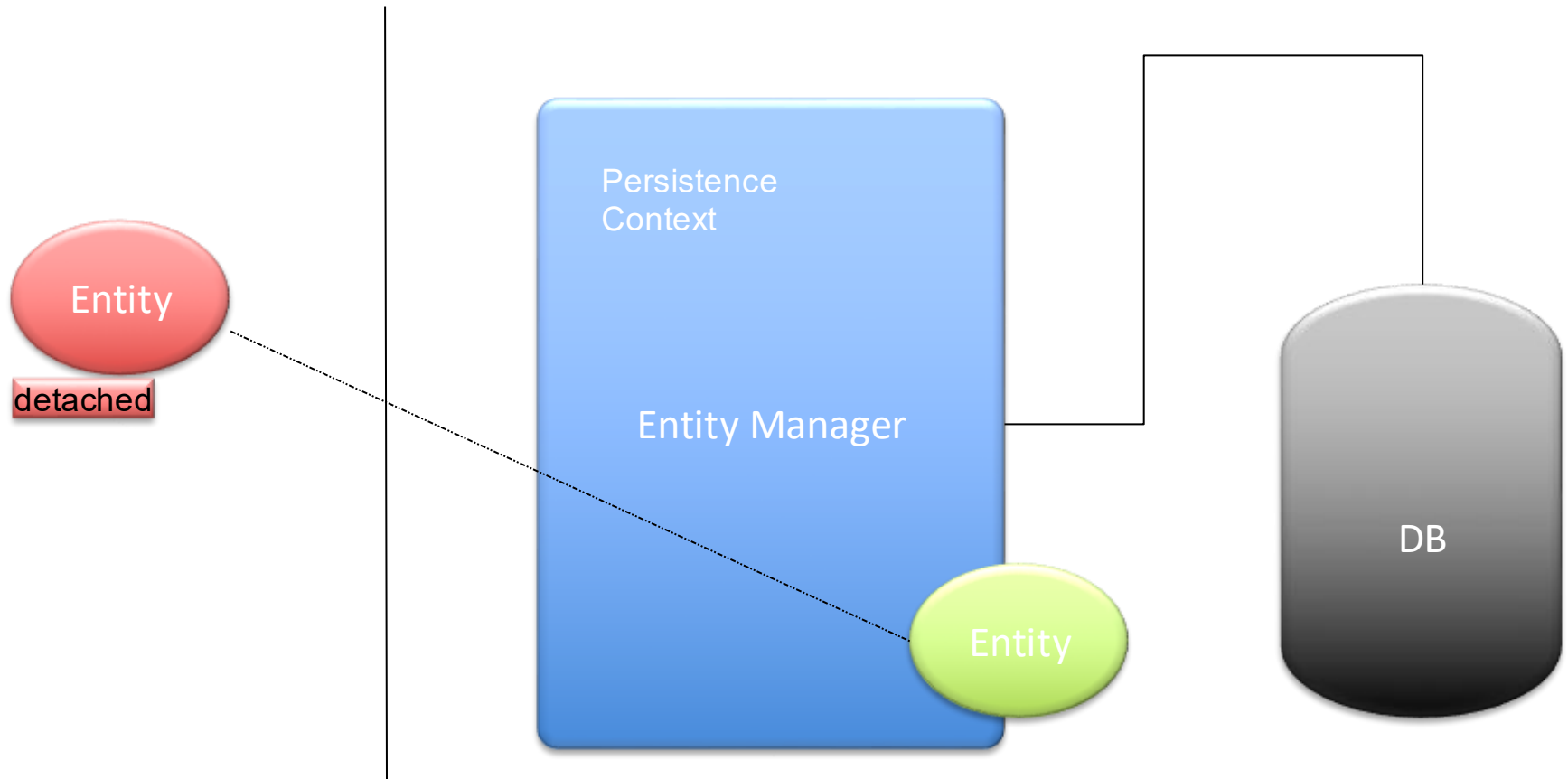
Lazy Loading und Fetch Joins

```
@OneToMany(fetch = FetchType.LAZY)  
private List<IArticleItem> articleItems;
```

```
Query query = entityManager  
    .createQuery("SELECT b FROM Basket b,"  
    + " LEFT JOIN FETCH b.articleItems");
```

Detached Relations

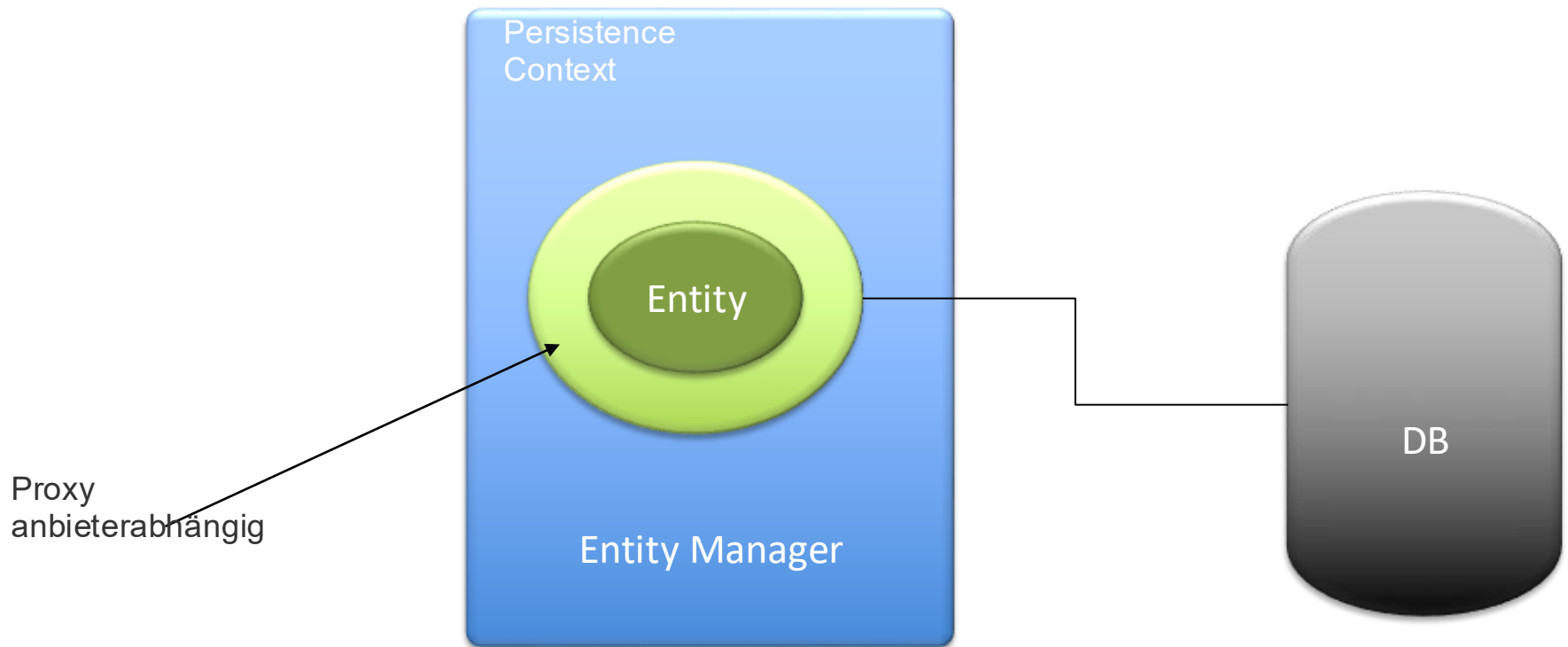
- Attached / Detached



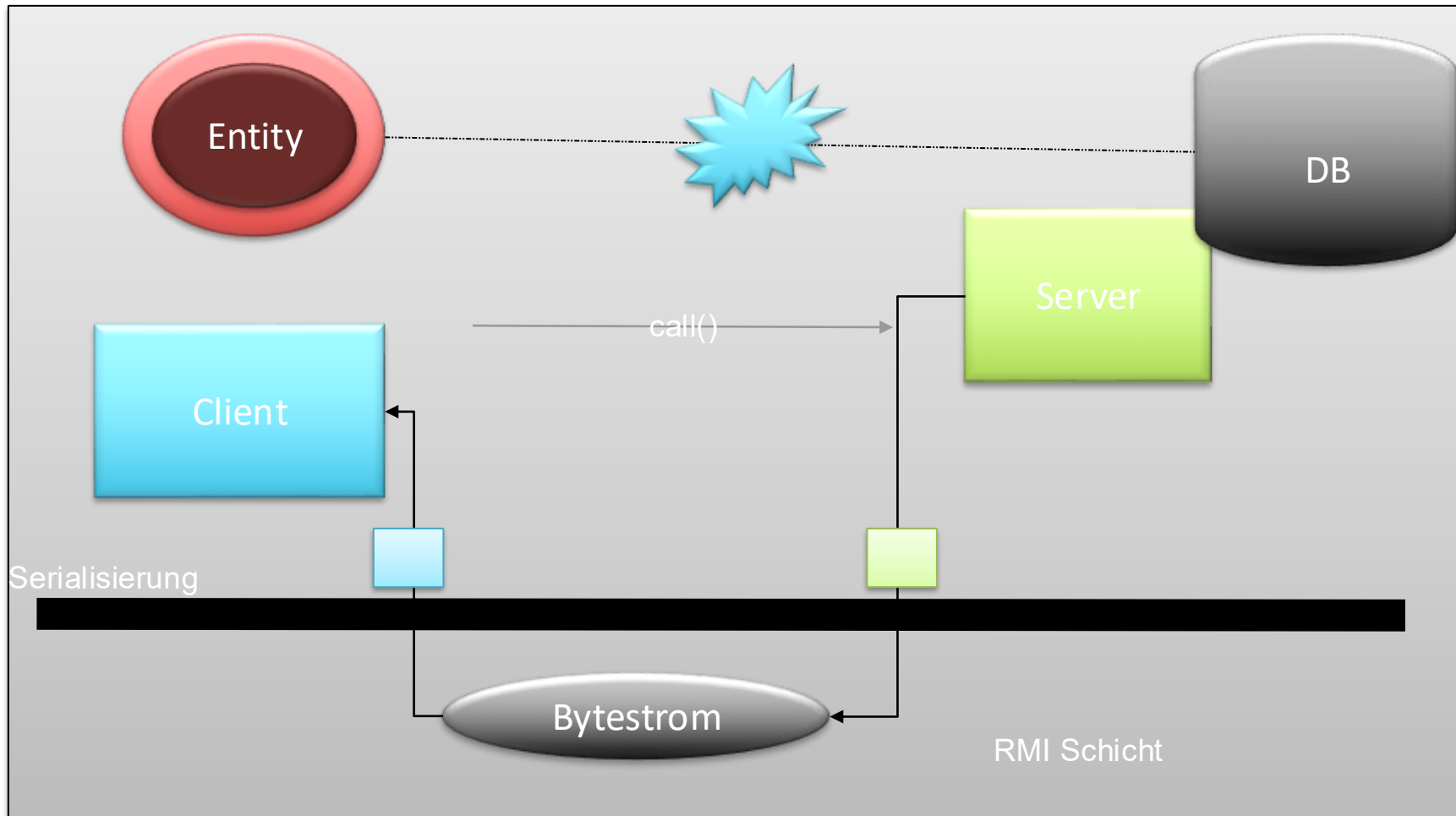
Entity Relationship

Lazy & Eager

- `@OneToMany (fetch=FetchType.EAGER)`



Detached Lazy



Funktionen

<i>Funktion</i>	<i>Beschreibung</i>
AVG	Mittelwertsberechnung
COUNT	Berechnet die Anzahl Element in der Gruppe
MIN, MAX	Bestimmt das (Min-)Maximum der Elemente in der Gruppe
SUM	Bildet die Summe der Elemente in der Gruppe
ABS	Absolute Wert einer Zahl
SQRT	Wurzel
MOD	Modulo
LOWER,UPPER	Kleinbuchstaben, Großbuchstaben
TRIM	Abschneiden eines bestimmten Zeichens am Anfang und Ende
CONCAT	Konkationation zweier Zeichenketten
LENGTH	Länge einer Zeichenkette
LOCATE	Position eines Zeichens in einer Zeichenkette
SUBSTRING	

Gruppierungen

```
Query query = entityManager
    .createQuery("SELECT c.name, COUNT(items) "
        + "FROM Customer AS c, IN(c.basket.articleItems) " + "items"
        + "GROUP BY c.name");

Query query = entityManager
    .createQuery("SELECT c.name, COUNT(items) "
        + "FROM Customer AS c, IN(c.basket.articleItems) " + "items"
        + "GROUP BY c.name HAVING COUNT(items) > 5");
```

Aufgabe 2_08: Teil 1

- Die Geschäftsführung möchte eine Statistik darüber, wie oft die verschiedenen Fahrzeugmodelle reserviert wurden. Dabei interessieren uns besonders die Modelle, die eine hohe Auslastung haben.
- **Aufgabe** Testklasse `TestQuery#testGroupBy()`.
 - Erstelle eine JPQL-Abfrage, die den Modellnamen des Fahrzeugs und die Anzahl der Reservierungen für dieses Modell zurückgibt.
 - Du musst dafür über Customer und deren reservations navigieren.
 - Das Ergebnis der Abfrage ist eine `List<Object[]>`. Jedes `Object[]` enthält an Index 0 den Modellnamen (String) und an Index 1 die Anzahl (Long).



Aufgabe 2_08: Teil 2

- Gruppierung mit Filterung (HAVING): Implementiere die Methode `testGroupByAndHaving()`.
 - Erweitere die vorherige Abfrage. Wir möchten nur die Modelle sehen, die mehr als 2 Reservierungen haben.
 - Nutze dafür die HAVING-Klausel.



Aufgabe 2_08: Erfolgskriterien

- Beide Testmethoden laufen grün.
- Du hast verstanden, dass im SELECT bei einer Gruppierung nur Felder stehen dürfen, nach denen auch gruppiert wird (oder die in einer Aggregatfunktion stehen).
- Du kannst den Unterschied zwischen WHERE und HAVING erklären.





Kleines Quiz

1. Wenn du `SELECT count(res) FROM Reservation res` ausführst, was ist der Rückgabetypp von `getSingleResult()`? Ist es ein Integer, ein Long oder ein int?
2. Könnte man die Bedingung `HAVING count(res) > 2` auch einfach in die `WHERE`-Klausel schreiben? Warum (nicht)?

Unterabfragen

```
Query query = entityManager
    .createQuery("SELECT c.name FROM Customer AS c"
        + "WHERE 5 < (SELECT COUNT(it) FROM"
        + "c.basket.items it)");
```

Aufgabe 2_09: Teil 1

- In unserem Autovermietungssystem (Rent-a-Car) möchten wir gezielte Analysen über das Buchungsverhalten unserer Kunden durchführen. Ein wichtiger Aspekt ist die Identifizierung von "Premium-Kunden", die bereits einen signifikanten Umsatz generiert haben. Da die Umsatzdaten (Preise) in den einzelnen Reservierungen gespeichert sind, müssen wir diese für jeden Kunden aggregieren und filtern.
- In dieser Übung lernst du, wie du Subqueries (Unterabfragen) in JPQL einsetzt, um komplexe Filterbedingungen zu formulieren, die auf aggregierten Daten von verknüpften Entitäten basieren



Aufgabe 2_09: Teil 2

- Ergänzen Sie die Testmethode testSubQuery in der Klasse TestQuery, um folgende Abfrage mittels JPQL zu implementieren:
- **Ziel:** Finden Sie alle Kunden (Customer), deren Gesamtsumme aller Reservierungspreise einen bestimmten Schwellenwert überschreitet.
- **Anforderung:** Verwenden Sie eine JPQL-Abfrage mit einer Subquery.
- Die Subquery soll die Summe (SUM) der Preise (price) aller Reservierungen (reservations) berechnen, die einem Kunden zugeordnet sind.
- Filtern Sie die Kunden so, dass nur diejenigen zurückgegeben werden, bei denen diese Summe größer als 200 ist.
- Das Ergebnis soll eine Liste von Customer-Objekten sein.



Aufgabe 2_09: Erfolgskriterien

- Korrekte JPQL-Syntax: Die Abfrage verwendet eine korrekte Unterabfrage im WHERE-Teil.
- Aggregatfunktion: In der Subquery wird die SUM()-Funktion korrekt auf das Preisfeld der Reservierungen angewendet.
- Filterung: Es werden nur Kunden zurückgegeben, deren kumulierter Reservierungswert über 200 liegt.
- Test-Pass: Die Assertion assertEquals(2, resultList.size()) wird erfolgreich ausgeführt (basierend auf den in setUp vorbereiteten Testdaten).
- Typisierung: Die Ergebnismethode liefert die erwartete Liste von Entitäten zurück.

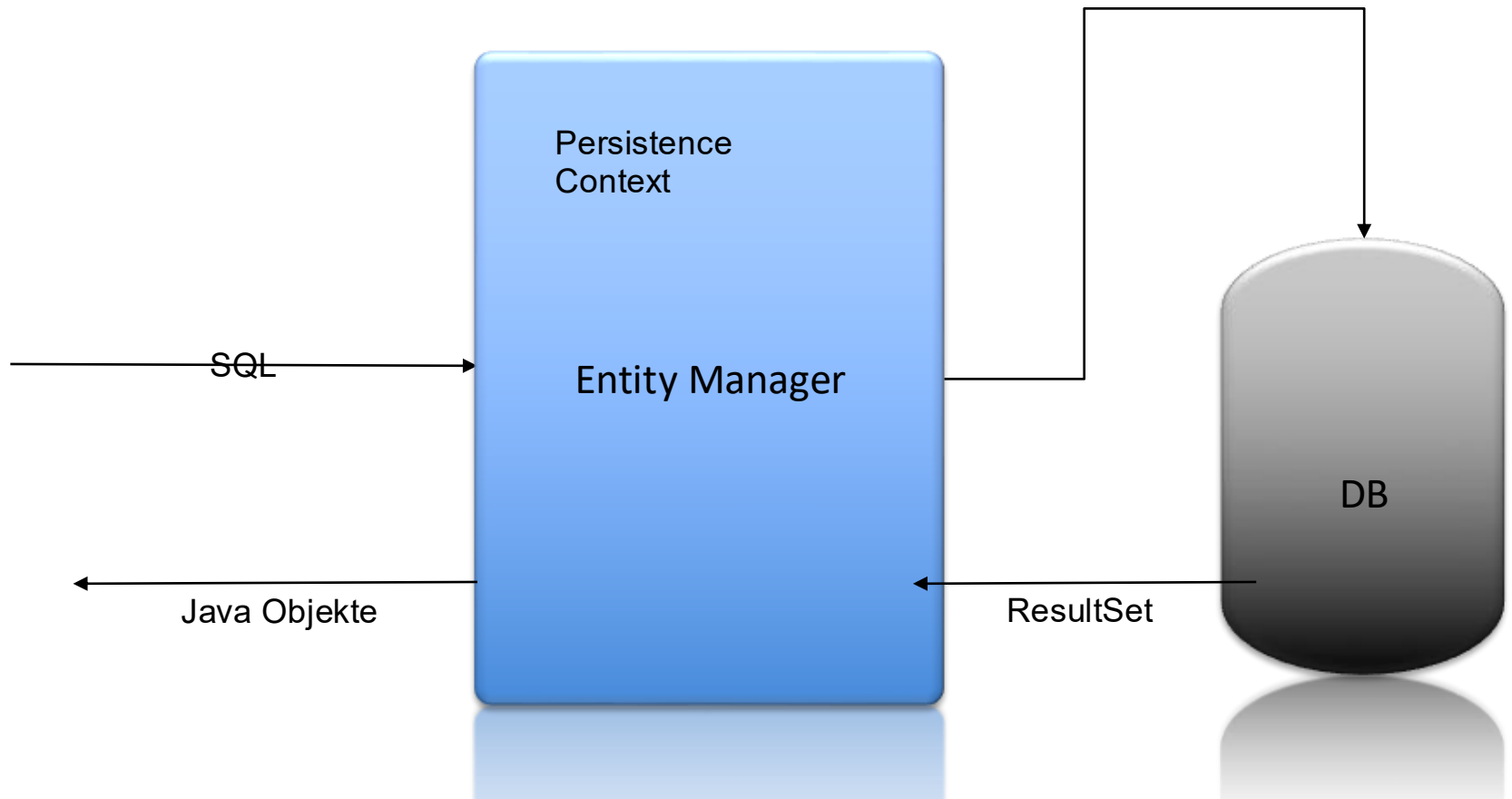




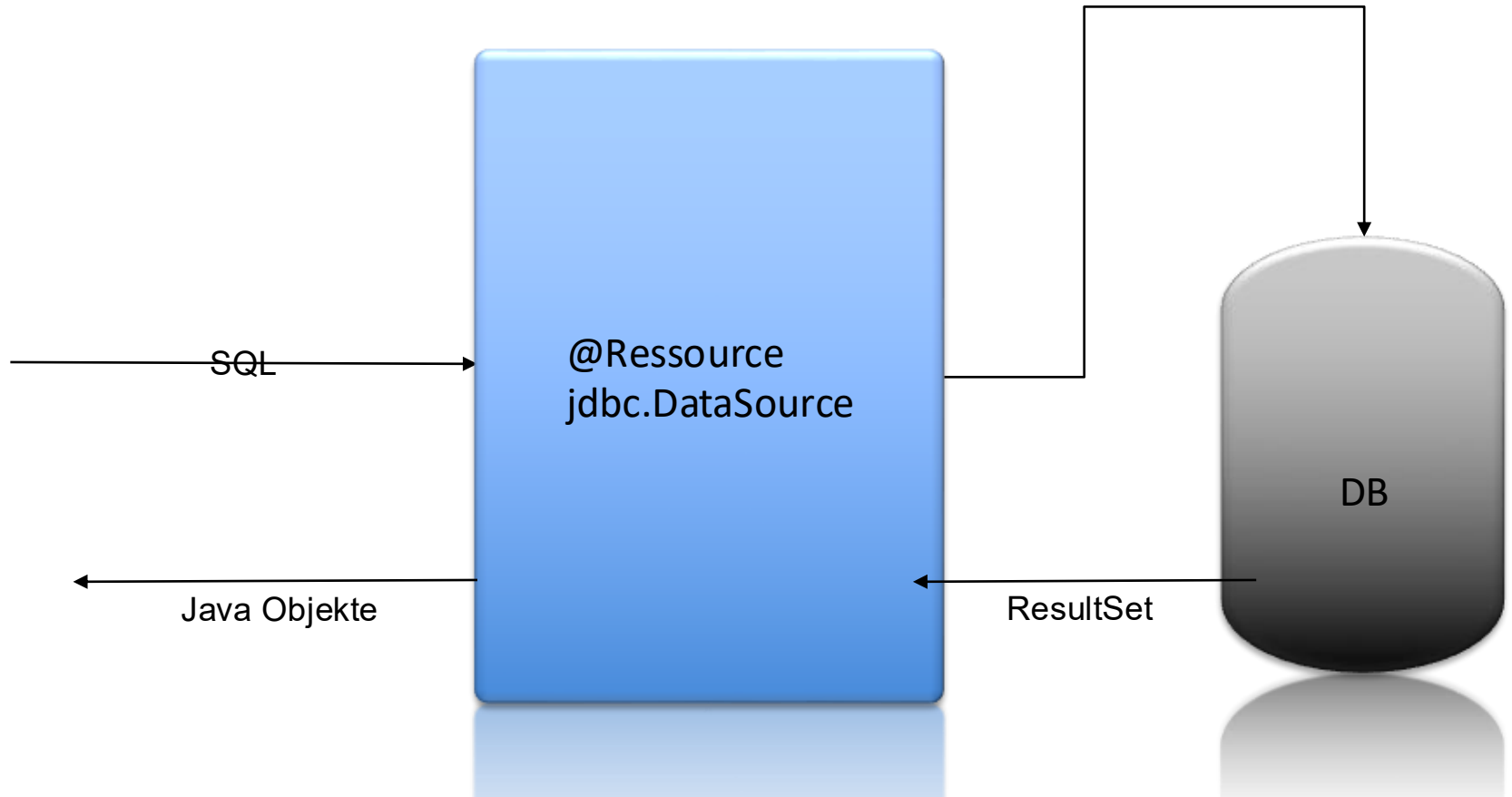
Kleines Quiz

1. Was ist der entscheidende Unterschied in der Formulierung einer Subquery, wenn man sie auf eine Collection-Variable des äußeren Objekts (wie im Beispiel k.reservations) bezieht, im Vergleich zu einer völlig unabhängigen Entität?
 - a) Hintergrund: Denken Sie an die Identifizierung des Objekts in der Subquery. Wie "weiß" die Subquery, welche Reservierungen sie summieren soll?
2. In welchen Teilen einer JPQL-Abfrage dürfen Subqueries laut JPA-Standard verwendet werden?
 - a) Nur in der WHERE-Klausel.
 - b) Nur in der WHERE- und HAVING-Klausel.
 - c) In der SELECT-, WHERE- und HAVING-Klausel.
 - d) In allen Klauseln, inklusive der FROM-Klausel

Native Query



Native Query 2

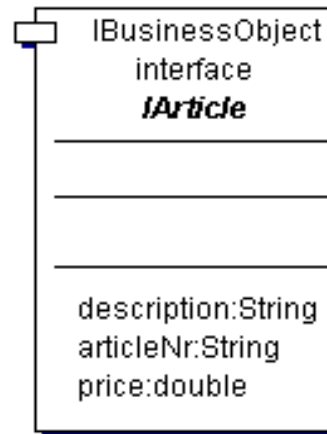


Entity Manager

EntityManager API - Query

- Query createQuery(String queryString)
- Query createNamedQuery(String name)
- **Query createNativeQuery(String sqlString)**
- **Query createNativeQuery(String sqlString, String resultSetMapping)**
- **Query createNativeQuery(String sqlString, Class resultClass)**

Native Queries mit ResultClass



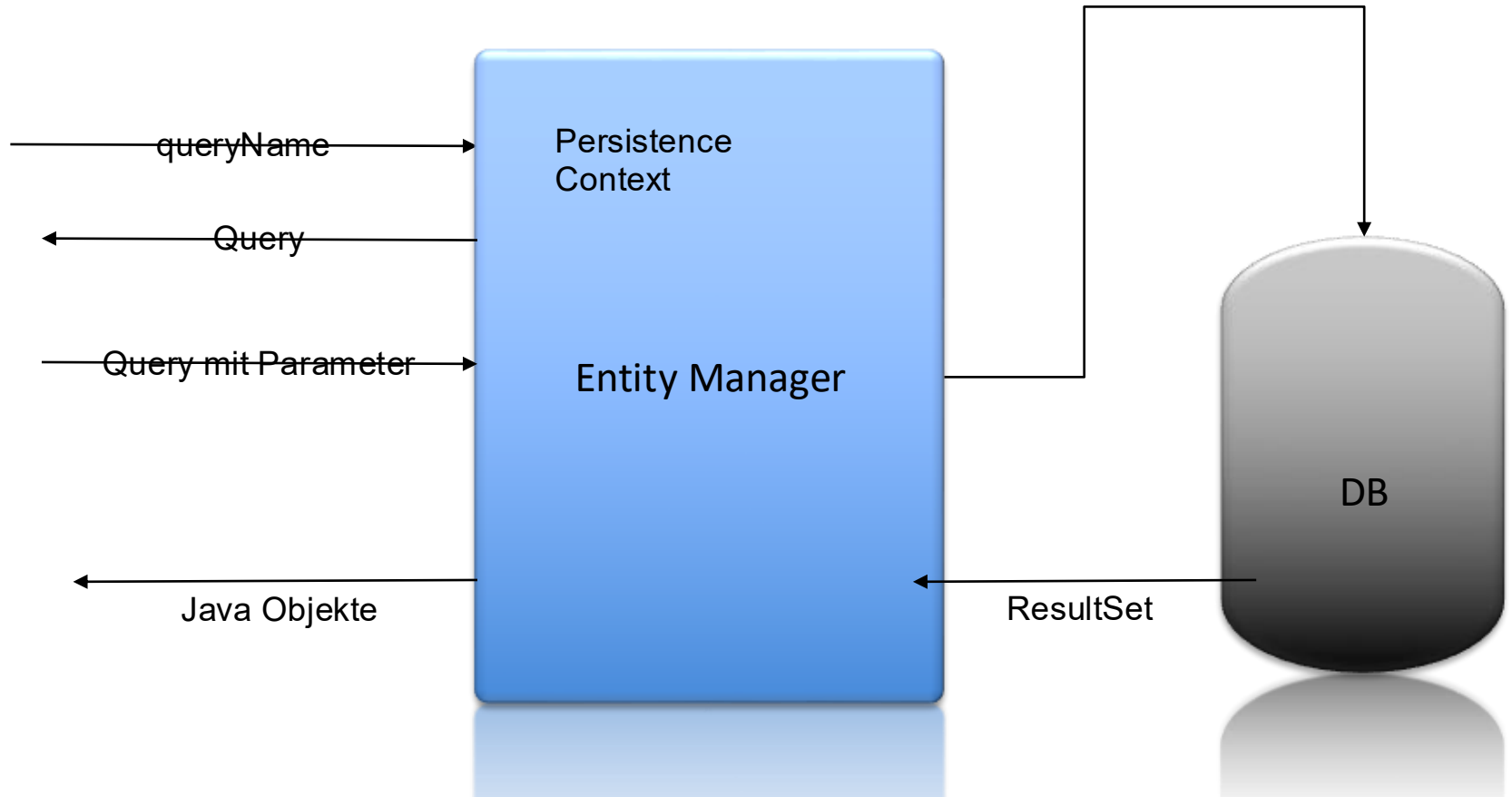
```
Query query = entityManager
    .createNativeQuery("SELECT a.id, a.description,
        " "a.articelNr, a.price FROM TBL_Article a",
        Article.class);
```

Native Queries mit ResultSetMapping

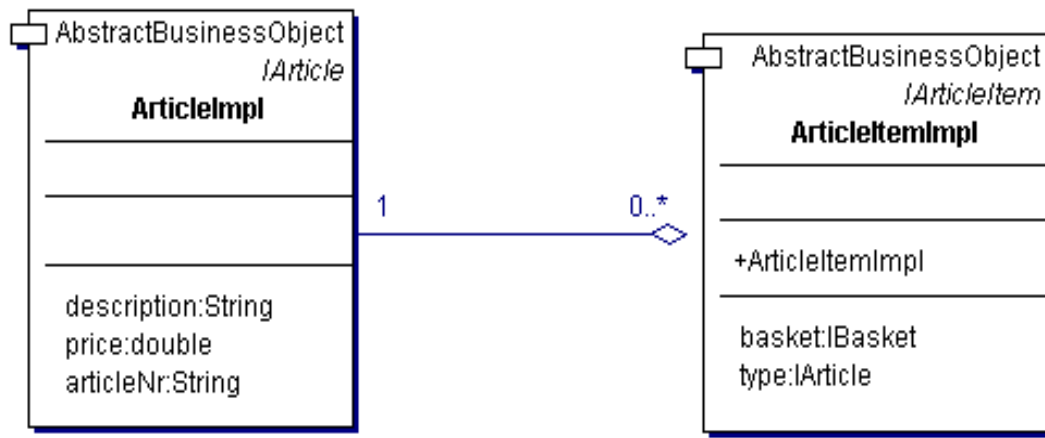
```
@Entity
@SqlResultSetMapping(name =
    "articleItemAndArticleMapping",
    entities={@EntityResult(entityClass= Article.class),
        @EntityResult(entityClass= ArticleItem.class,
            fields={@FieldResult(name= "id",column= "IT_ID")
                }}})
```

```
Query query = entityManager
    .createNativeQuery("SELECT it.id AS IT_ID,
        it.basket_id, " + "it.article_id, a.id, a.articelNr
        FROM TBL_Article a, " + "TBL_ArticleItem it WHERE
        it.article_id = a.id",
        "articleItemAndArticleMapping");
```

Named Query



Named Query



```
@NamedQuery (name =  
    "getArticleItemsByArticle",  
    query = "FROM ArticleItem it WHERE it.type = " +  
            ":article")
```

Criteria API

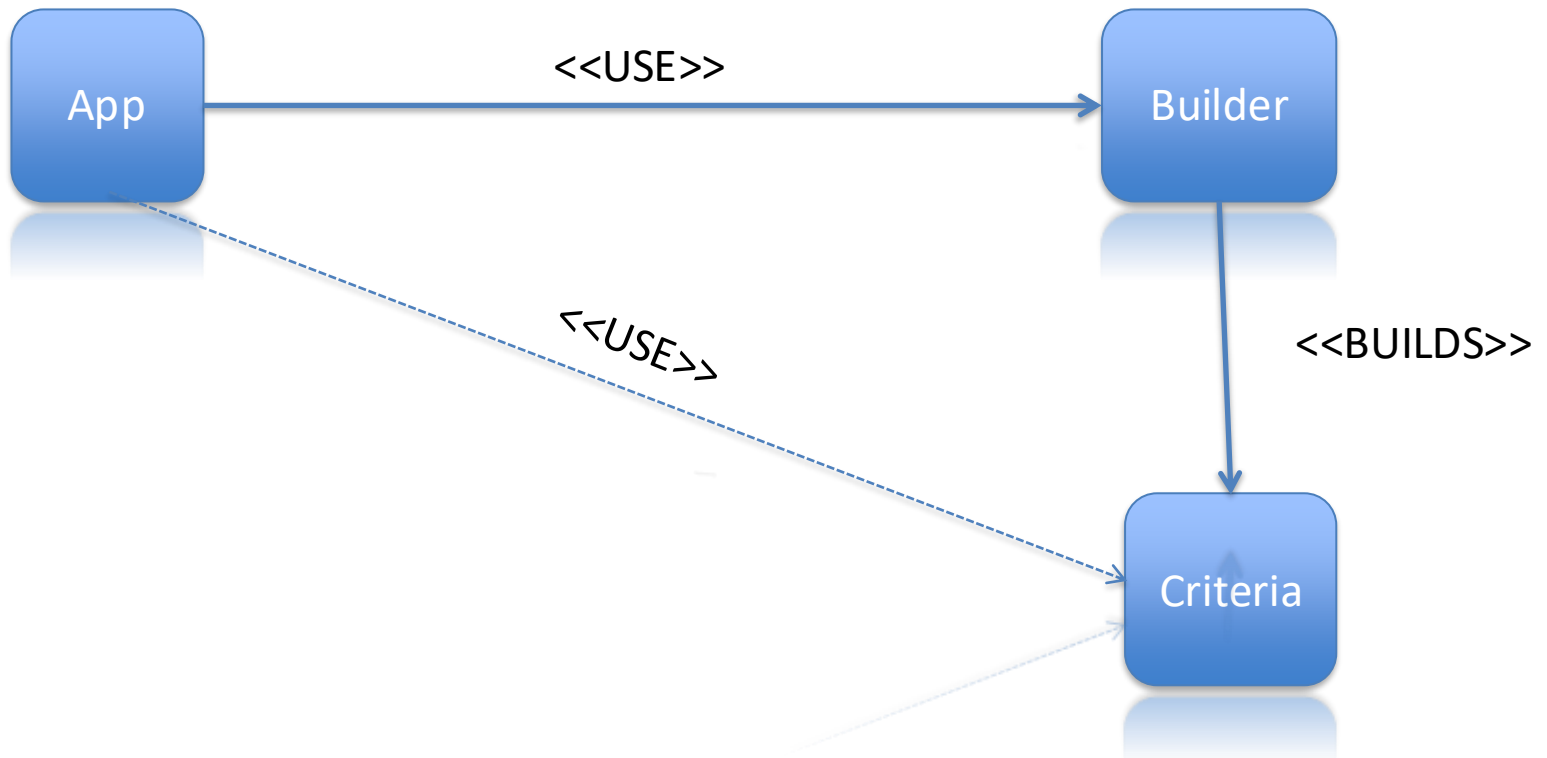
JPQL ist
mächtig, aber:

- Strings → **keine Typsicherheit**
- Fehler erst zur Laufzeit
- Dynamische Abfragen werden schnell unübersichtlich

Criteria API:

- **Typsicher**, objektorientiert
- Ideal für **dynamische Filter**, die zur Laufzeit entstehen
- Bessere Unterstützung für komplexe Query-Generierung

CRITERIA API



```
CriteriaQuery<Basket> queryBaskets = criteriaBuilder.createQuery(Basket.class);
```

```
Root<Basket> baskets= queryBaskets.from(Basket.class);  
queryBaskets.select(baskets);
```

```
List<Basket> resultList = manager.createQuery(queryBaskets).getResultList();
```

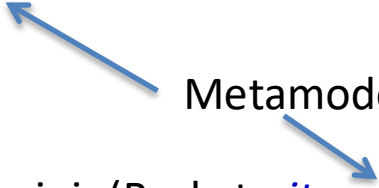
```
//SELECT b from Basket b
```

JPQL ist kompakter, Criteria API ist **typsicher und dynamisch**.

Aufbau einer Query

Path<BigDecimal> priceAttribute = root.get(Basket_.*price*)
//without Metamodel root.get(*"price"*)

Metamodel
baskets.join(Basket_.*items*);



- Typsicherheit
- Autocomplete
- Weniger Fehler
- Annotation Processing.

Metamodel

```
Path<String> nameAttribute = itemsOfBasket.get(ArticleItem_.name)
```

```
Predicate nameEqualsIPAD = criteriaBuilder.equal(nameAttribute, "IPad");
```

```
queryBaskets.select(baskets).where(nameEqualsIPAD).  
    distinct(true);
```

WHERE

```
CriteriaQuery<Basket> queryBaskets =  
criteriaBuilder.createQuery(Basket.class);
```

```
Root<Basket> root= queryBaskets.from(Basket.class);
```

```
queryBaskets.select(root)  
    .where(builder.gt(root.get("price"), 100));
```

```
SELECT root FROM Basket root WHERE root.price > 100
```

Takeway: JPQL ist kompakter, Criteria API ist **typsicher und dynamisch**.

JPQL vs Criteria

```
Path<String> nameAttribute = itemsOfBasket.get(ArticleItem_.name)
```

```
Predicate nameEqualsIPAD = criteriaBuilder.equal(nameAttribute, "IPad");
```

```
queryBaskets.select(baskets).where(nameEqualsIPAD).  
    distinct(true);
```

- cb.and(...)
- cb.or(...)
- cb.not(...)

WHERE

Kombination

```
CriteriaQuery<Person> query = cb.createQuery(Person.class);  
Root<Person> root = query.from(Person.class);
```

```
Predicate olderThan18 = cb.gt(root.get("age"), 18);
```

```
Predicate nameStartsWithA = cb.like(root.get("name"), "A%");  
query.where(cb.and(olderThan18, nameStartsWithA));
```

- cb.and(...)
- cb.or(...)
- cb.not(...)

Aggregation

```
CriteriaQuery<Long> query =  
cb.createQuery(Long.class);
```

```
...
```

```
query.select(cb.count(root))  
    .where(cb.and(older18 , nameWithA));
```

```
---
```

- count, sum, avg, min, max
- groupBy, having

Typische Fehler

- Vergessen, `select(root)` zu setzen
- Falsche Property-Namen bei `root.get("...")`
- Zu komplexe Queries → lieber QueryDSL oder Spring Data Specifications
- Metamodel nicht generiert (Annotation Processing fehlt)



Wann CriteriaAPI

Gut geeignet für:

- Dynamische Filter
- Komplexe, programmatisch erzeugte Queries
- Frameworks (z. B. Spring Data Specifications)



Weniger geeignet für:

- Einfache statische Queries → JPQL ist lesbarer
- Sehr komplexe Query-Logik → QueryDSL oft eleganter

Aufgabe 2_15: Überblick

- Du findest im Modul `2_15_CriteriaAPI_Simple` die Testklasse `net.rentacar.TestQuery`. Diese enthält drei leere Testmethoden, die du mit Hilfe der Criteria API implementieren sollst. Die fachlichen Anforderungen entsprechen denen aus der Übung `2_04_JPQL_WhereClause`, jedoch soll hier kein JPQL-String, sondern das Criteria-Objektmodell verwendet werden.



Aufgabe 2_15: Einzelaufgabe

- `testQueryForNutzerWithPersonnameISMichaelOrMathias`
 - Frage alle User ab, deren zugehörige Person den Vornamen "Michael" oder "Mathias" hat.
 - Verwende den `in()` Operator auf dem Attributpfad `person.firstName`.
 - **Erwartung:** 2 Ergebnisse.
- `testQueryForShopMitKeinemVehicle`
 - Finde alle Shop-Objekte, die aktuell kein Fahrzeug im Fuhrpark (carpool) haben.
 - Nutze dazu die Methode `cb.isEmpty()` des `CriteriaBuilder`.
 - **Erwartung:** 2 Ergebnisse.
- `testQueryForShopsichtInStuttgartUndMuenchen_MitINOperator`
 - Suche alle Shops, deren Standort (location) **nicht** in "Muenchen" oder "Stuttgart" liegt.
 - Verwende eine Kombination aus `cb.not()` und dem `in()` Operator auf dem Feld `location`.
 - **Erwartung:** 1 Ergebnis.
- **Tipps**
 - Nutze `manager.getCriteriaBuilder()`, um den Builder zu erhalten.
 - Erstelle eine `CriteriaQuery<T>` für den gewünschten Ergebnistyp.
 - Definiere den Root-Pfad mit `cq.from(Entity.class)`.
 - Denke daran, die Abfrage mit `manager.createQuery(cq).getResultList()` auszuführen.





Zusatz

- `testQueryForUserByLastName`
 - Suche nach dem User, dessen Person den Nachnamen "Mayer" trägt.
 - Nutze `user.get("person").get("lastName")`.
 - **Erwartung:** 1 Ergebnis.
- `testCountAllShops`
 - Ermittle die Gesamtzahl aller registrierten Shops.
 - Verwende `cb.count()` und als Ergebnistyp der Query `Long.class`.
 - **Erwartung:** 3 Ergebnisse (Long-Wert 3).

Zusammenfassung



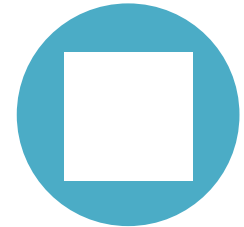
**TYPESICHERE,
DYNAMISCHE
QUERY-GENERIERUNG**



FÜR FLEXIBLE FILTER



**ETWAS VERBALER ALS
JPQL**

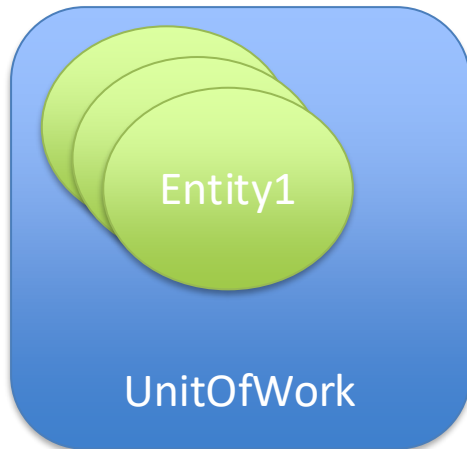


**METAMODEL STEIGERT
TYPESICHERHEIT UND
IDE-SUPPORT**



EJB und Transaktionen

UnitOfWork / ACID



- ACID
 - Atomic (All or Nothing))
 - Consistent
 - Isolated (Serializable)
 - Durable

EJB Transaction Management

Deklaratives Transaktionsmanagement

- Trennung von Business Code

Transaction Scope (Unit-of-work)

- Beteiligt: Session und Entity Beans
- Startet mit Aufruf einer Business Methode
- Propagierung zu EntityManager und EJB Referenzen

Transaktion Committed

- Alle Tasks innerhalb des Transactions Scopes erfolgreich (Atomic)

Ende der Transaktion

- Exception

Attribute der Transaktion

- @TransactionAttribute
 - NotSupported
 - Supports
 - Required (default)
 - RequiresNew
 - Mandatory
 - Never
- Verwendung
 - Klassenebene (SessionBean) als Default für alle Methoden
 - Methodenebene

Beispiel

- Klassenebene

```
@Stateless
```

```
@TransactionAttribute(TransactionAttributeType.REQUIRED)
```

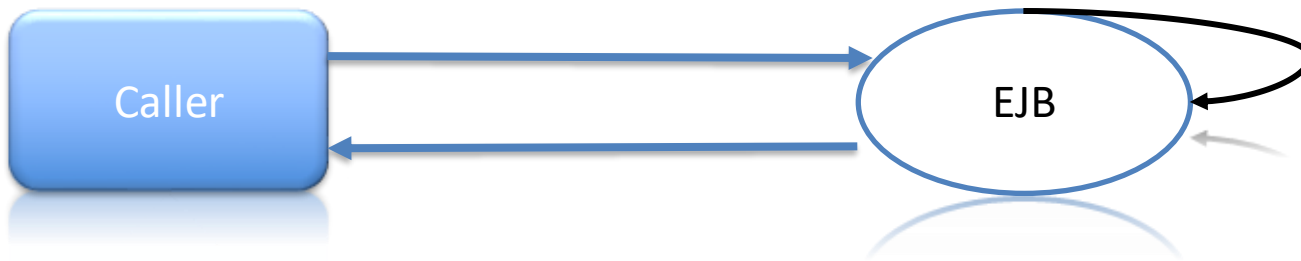
```
public class MyStoreServiceImpl implements  
    MyStoreServiceRemote {
```

- Methodenebene

```
@TransactionAttribute(TransactionAttributeType.REQUIRES\_NEW)
```

```
public IArticle insertArticle(IArticle article) {  
    entityManager.persist(article);
```

NotSupported

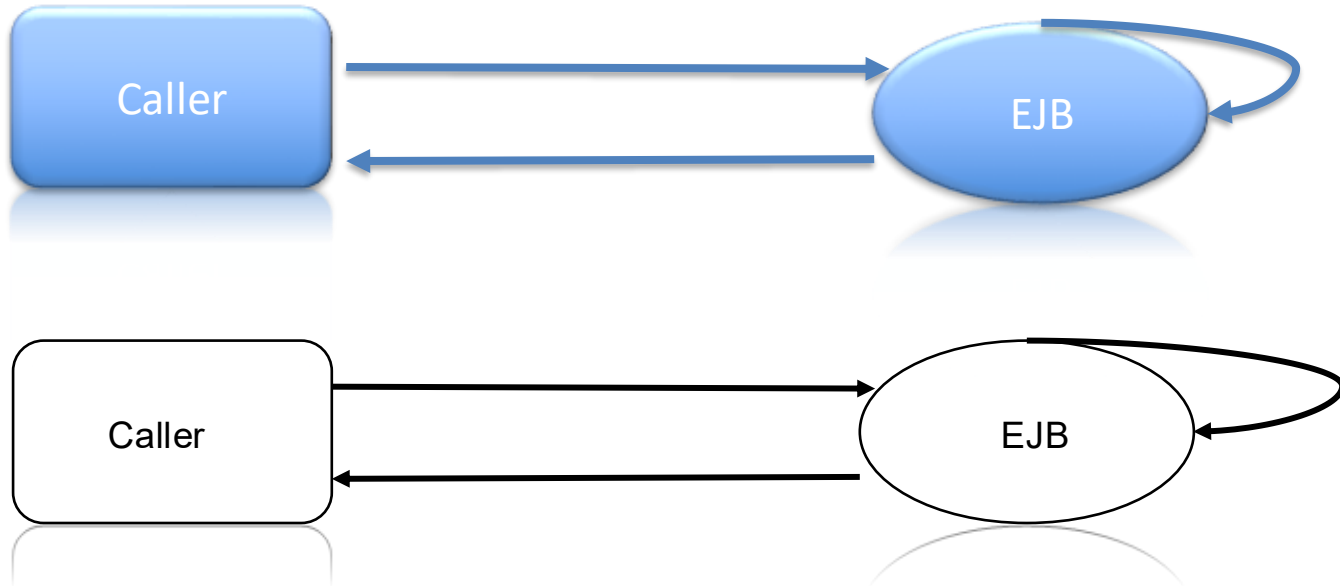


Transaction



Transactionsfree

Supports

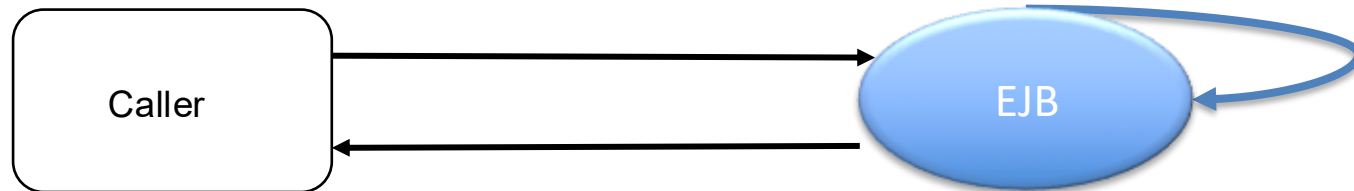
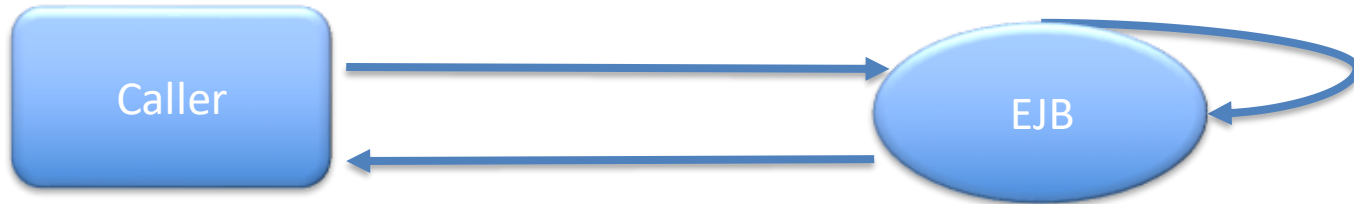


Transaction



Transactionsfree

Required

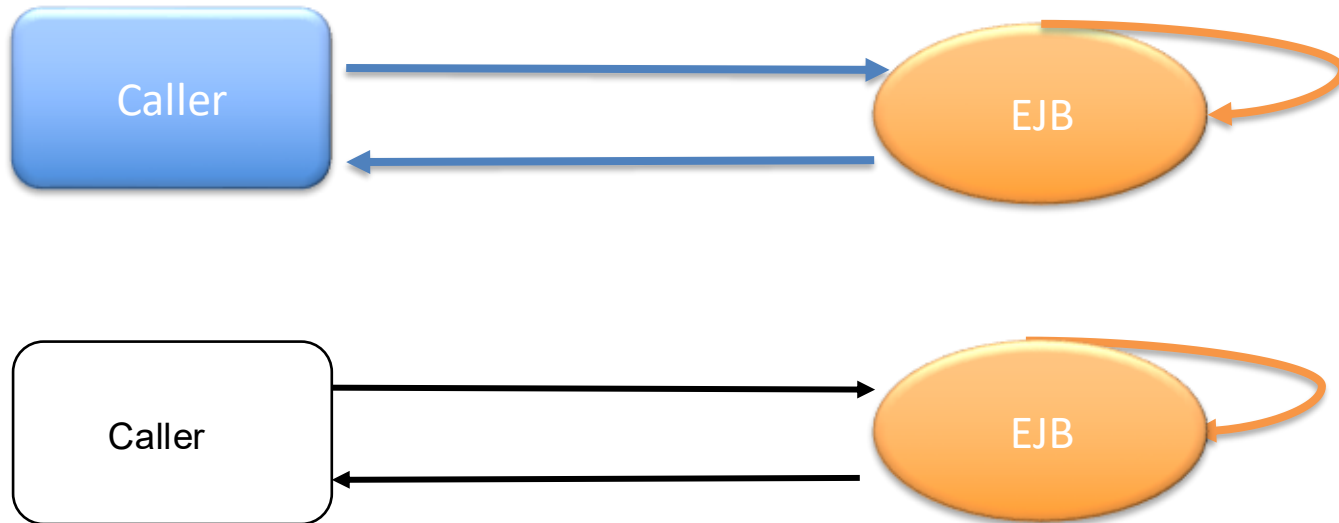


Transaction



Transactionsfree

RequiresNew



Transaction Caller

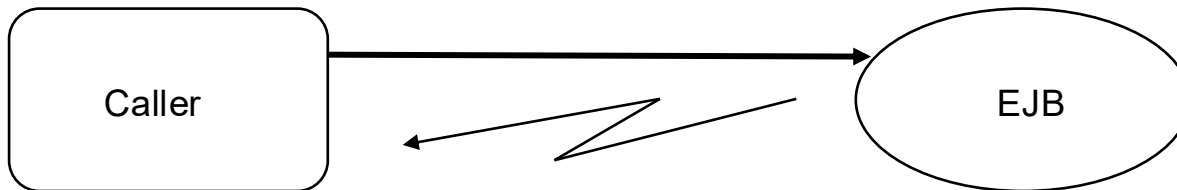
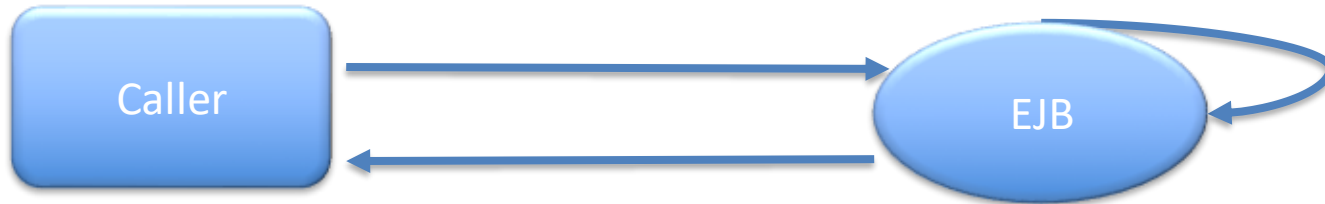


Transactionfree



Transaction Called Object

Mandatory

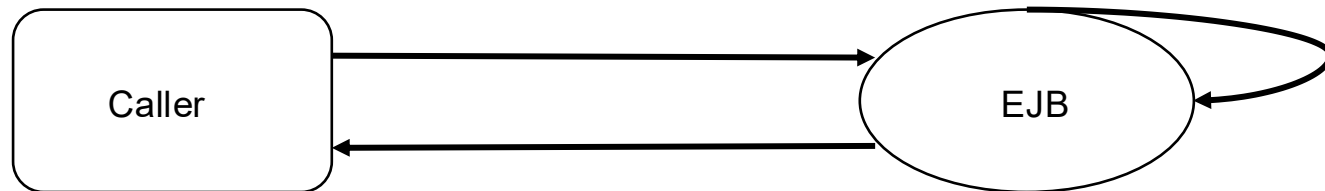


Transaction



Transactionsfree

Never



Transaction



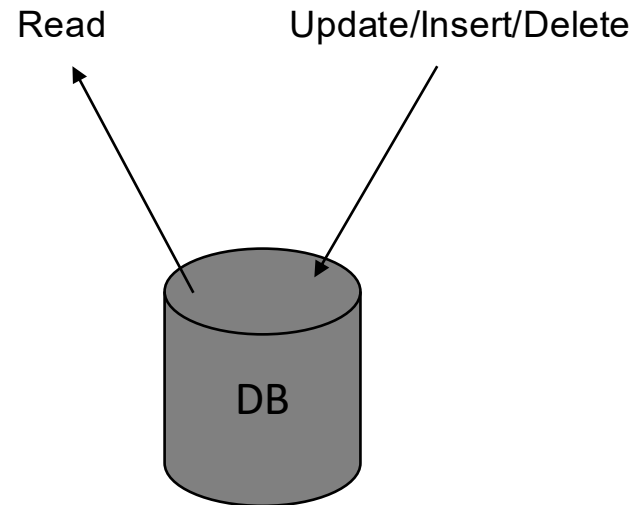
Transaktionsfree

EntityManager, MDBs und Transaktionen

- EJB Spezifikation
 - Aufruf des EntityManagers innerhalb eines Transaktionskontextes
 - Required, RequiresNew, Mandatory
- Message Driven Bean
 - Wird durch MessageBroker aufgerufen
 - Kein Client/EJB Transaktion
 - NotSupported, Required

Isolation

- Dirty Reads
 - Schreiben der 1. Transaktion
 - Lesen der 2. Transaktion
 - Rollback der 1. Transaktion
- Non-Repeatable Reads
 - Zweimaliges Lesen einer Transaktion
 - Gleiche Daten
 - unterschiedliches Ergebniss
- Phantom Reads
 - Nicht mehr existent
 - Änderung d. 2. Transaktion

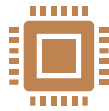


Isolation Levels



Read Uncommitted

Dirty Reads, Nonrepeatable Reads, Phantom Reads



Read Committed

Erlaubt: Lesen von Daten im „committed“ Zustand
Nonrepeatable Reads,
Phantom Reads



Repeatable Read

Keine Änderung an Daten,
die durch 1. Transaktion
gelesen
Phantom Reads



Serializable

Exklusives Schreiben und
Lesen
Sehr restriktiv

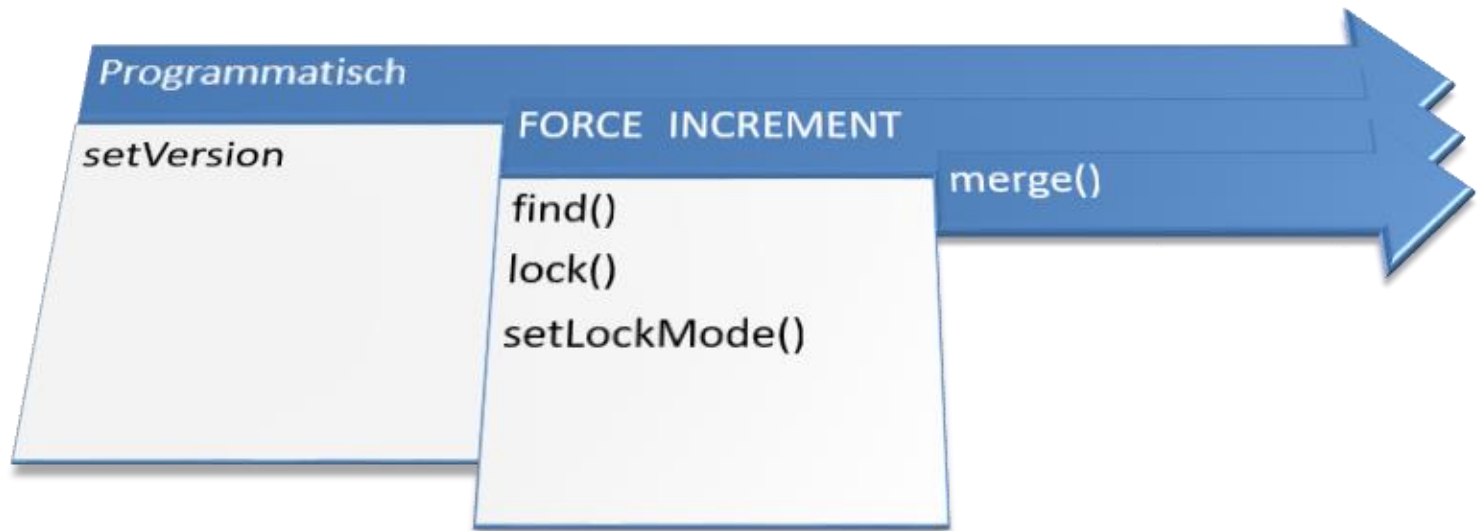
Performance vs. Konsistenz

- Serializable
 - Keine Nebenläufigkeit bei DB-Zugriffe
 - Lesen?
 - Aber IMMER konsistente Daten
- Wahl des Isolation Level
 - Anwendungsabhängig
 - Datenabhängig
 - Effekte auf Integrität der Anwendung?

LockMode

- EntityManager
 - lock() Methode
 - Subjekt: Entity
- LockModeType.OPTIMISTIC
 - Dirty Reads, Nonrepeatable Reads
- LockModeType.OPTIMISTIC_FORCE_INCREMENT
 - Version Attribut inkrementiert

```
@Version  
private Integer version;
```



SperrTypen

PESSIMISTIC_READ

- Geteilte Sperre

PESSIMISTIC_WRITE

- Schreib-Sperre
- Entspricht einer exklusiven Sperre mittels DB Lock auf Zeile(n)

PESSIMISTIC_FORCE_INCREMENT

- Exclusive Sperre
- Zusätzlich wird das Version-Attribut inkrementiert

Lock Scope

Verbundhierarchien

@SecondaryTable

Alle in Beziehung stehenden Entities gelockt

- Fremdschlüssel in Tabelle der Entity gehalten.

javax.persistence.lock.scope =
PessimisticLockScope.EXTENDED

- Alle in Beziehung stehenden Entites, die über Join-Tables gemappt sind.

Bean Managed Transaction

- `@TransactionManagement(TransactionManagerType.BEAN)`
- Erzeugung
 - SessionContext erzeugt Transaktion
 - @Resource Dependency Injection der Transaktion
- Stateful Session Beans
 - Methode des Beginn und Ende der Transaktion können differieren
- MDB
 - Begin und Ende der Transaktion in `onMessage()`

UserTransaction API

- `begin()`
- `Commit()`
- `Rollback()`
- `setRollbackOnly()`
- `setTransactionTimeout(int seconds)`
- `getStatus()`

Container Managed Transaction

- SessionContext
 - getRollbackOnly()
 - setRollbackOnly()

Ausnahmen und Transaktionen

SystemException

- Alle RuntimeExceptions und Unterklassen
- Alle RemoteExceptions und Unterklassen
- Führen zum Rollback, Logeintrag, "EJB sterben"

ApplicationException

- Alle NonRuntimeException
- @ApplicationException
- Kein Rollback

EJBTransactionRollbackException

- Client Seite hat Transaktion gestartet

@ApplicationException(rollback=true)